

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Информационные технологии
и прикладная математика»

Лабораторные работы по курсу
«Численные методы»

Выполнил: Арусланов К. А.

Преподаватель: Ревизников Д. Л.

Вариант: 1

Оценка:

Москва, 2026

Оглавление

Лабораторная работа №1	3
О программе.....	3
Алгоритм LU-разложения матрицы	3
Метод прогонки.....	7
Итерационные методы решения СЛАУ	10
Метод вращений	14
QR-алгоритм нахождения собственных значений матриц	17
Лабораторная работа №2	22
О программе.....	22
Решение нелинейных уравнений.....	22
Решение систем нелинейных уравнений	26
Лабораторная работа №3	31
О программе.....	31
Интерполяция	31
Сплайн	35
Метод наименьших квадратов	38
Численное дифференцирование	42
Численное интегрирование	44
Лабораторная работа №4	49
О программе.....	49
Численные методы решения задачи Коши	49
Численные методы решения краевой задачи для ОДУ	58

Лабораторная работа №1

О программе

Программа написана на языке C++ с использованием стандарта C++23. Решение реализовано модульно: для работы с матрицами был разработан класс `Matrix`, а отдельные численные методы вынесены в самостоятельные модули. В лабораторной работе реализованы методы LU-разложения, метод прогонки, итерационные методы решения СЛАУ, метод Якоби и QR-алгоритм.

Инструкция к запуску

Проект собирается с помощью CMake. Все входные данные хранятся в директории `input` в формате `txt`. При запуске программы последовательно выполняются все задания лабораторной работы, результаты сохраняются в директорию `output`.

Пример для выполнения всех заданий:

```
cmake --build build
./main
```

Алгоритм LU-разложения матрицы

LU-разложение матрицы представляет собой разложение матрицы A в произведение нижней и верхней треугольных матриц, т. е. $A = LU$, где L — нижняя треугольная матрица, U — верхняя треугольная матрица.

LU-разложение может быть построено с использованием метода Гаусса. Рассмотрим k -й шаг метода Гаусса, на котором осуществляется обнуление поддиагональных элементов k -го столбца матрицы $A^{(k-1)}$. С этой целью используется операция:

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - \mu_i^{(k)} a_{kj}^{(k-1)}, \text{ где } \mu_i^{(k)} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}; i = \overline{k+1, n}; j = \overline{k, n}$$

В терминах матричных операций такая операция эквивалентна умножению $A^{(k)} = M_k A^{(k-1)}$, где элементы матрицы M определяются следующим образом

$$m_{ij}^k = \begin{cases} 1, & i = j \\ 0, & i \neq j, j \neq k \\ -\mu_{k+1}^{(k)}, & i \neq j, j = k \end{cases}$$

В результате прямого хода метода Гаусса получим $A^{(n-1)} = U$,

$$A = A^{(0)} = M_1^{-1}A^{(1)} = M_1^{-1}M_2^{-1}A^{(2)} = M_1^{-1}M_2^{-1} \dots M_{n-1}^{-1}A^{(n-1)},$$

де $A^{(n-1)} = U$ — верхняя треугольная матрица, а $L = M_1^{-1}M_2^{-1} \dots M_{n-1}^{-1}$ — нижняя треугольная матрица, имеющая вид

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ \mu_2^{(1)} & 1 & 0 & 0 & 0 & 0 \\ \mu_3^{(1)} & \mu_3^{(2)} & 1 & 0 & 0 & 0 \\ \dots & \dots & \mu_{k+1}^{(k)} & 1 & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \mu_n^{(1)} & \mu_n^{(2)} & \mu_n^{(k+1)} & \dots & \mu_n^{(n-1)} & 1 \end{pmatrix}$$

В дальнейшем LU -разложение может быть эффективно использовано при решении систем линейных алгебраических уравнений вида $Ax = b$. Действительно, подставляя LU -разложение в СЛАУ, получим $LUx = b$, или $Ux = L^{-1}b$. Т.е. процесс решения СЛАУ сводится к двум простым этапам:

- 1) На первом этапе решается СЛАУ $Ly = b$. Поскольку матрица системы — нижняя треугольная, решение можно записать в явном виде:

$$y_1 = b_1, \quad y_i = b_i - \sum_{j=1}^{i-1} l_{ij}y_j, \quad i = \overline{2, n}$$

- 2) На втором этапе решается СЛАУ $Ux = y$ с верхней треугольной матрицей. Здесь, как и на предыдущем этапе, решение представляется в явном виде:

$$x_1 = \frac{y_n}{u_{nn}}, \quad x_i = \frac{1}{u_{ii}}(y_i - \sum_{j=i+1}^n u_{ij}y_j), \quad i = \overline{n-1, 1}$$

Зная LU -разложение легко также получить определитель матрицы A и обратную ей матрицу. Определитель находится как произведение элементов на главных диагоналях матриц L и U :

$$\det A = \prod_{i=1}^n l_{ii} \cdot u_{ii}$$

Обратную матрицу можно найти из отношения $AA^{-1} = LUA^{-1} = E$. Это уравнение также можно решить методом LU -разложения.

Входные данные:
$$\begin{cases} x_1 + 2 \cdot x_2 - 2 \cdot x_3 + 6 \cdot x_4 = 24 \\ -3x_1 - 5x_2 + 14x_3 + 13x_4 = 41 \\ x_1 + 2x_2 - 2x_3 - 2x_4 = 0 \\ -2x_1 - 4x_2 + 5x_3 + 10x_4 = 20 \end{cases}$$

Метод для LU-разложения:

```
LUDecomposition luDecompose(const Matrix& A_orig, std::ostream& log) {
    int n = A_orig.size();
    Matrix A = A_orig;                // рабочая копия
    Matrix L(n), U(n);
    std::vector<int> P(n);

    for (int i = 0; i < n; ++i) {
        P[i] = i;
        L(i, i) = 1.0;
    }

    log << "=== LU-разложение с выбором главного элемента ===\n";

    for (int k = 0; k < n; ++k) {
        // поиск главного элемента
        int pivot = k;
        double max_val = std::fabs(A(k, k));
        for (int i = k + 1; i < n; ++i) {
            double val = std::fabs(A(i, k));
            if (val > max_val) {
                max_val = val;
                pivot = i;
            }
        }

        log << "Шаг k=" << k << ": максимум в строке " << pivot
            << " (значение " << max_val << ")\n";

        if (max_val < EPS) {
            log << "ОШИБКА: матрица вырожденная!\n";
            throw std::runtime_error("Вырожденная матрица");
        }

        // перестановка
        if (pivot != k) {
            A.swapRows(k, pivot);
            std::swap(P[k], P[pivot]);
            log << "    - переставили строки " << k << " и " << pivot << "\n";
            for (int j = 0; j < k; ++j) { // только столбцы 0..k-1
                std::swap(L(k, j), L(pivot, j));
            }
            if (k > 0) {
                log << "    - также поменяли множители в L (строки "
                    << k << " <-> " << pivot << ")\n";
            }
        }

        // зануление
        for (int i = k + 1; i < n; ++i) {
            double factor = A(i, k) / A(k, k);
            L(i, k) = factor;
            A(i, k) = 0.0;
            for (int j = k + 1; j < n; ++j) {
                A(i, j) -= factor * A(k, j);
            }
            log << "    L[" << i << "][" << k << "] = " << factor << "\n";
        }
    }

    // заполняем U
    for (int i = 0; i < n; ++i)
        for (int j = i; j < n; ++j)
            U(i, j) = A(i, j);

    log << "LU-разложение завершено успешно\n";
    return {L, U, P};
}
```

```
}
```

Функция для решения СЛАУ с помощью LU-разложения:

```
std::vector<double> solveLU(const LUdecomposition& lu, const std::vector<double>& b) {
    int n = lu.L.size();
    std::vector<double> bp(n);
    for (int i = 0; i < n; ++i) bp[i] = b[lu.P[i]];    // Pb

    // прямой ход L y = Pb
    std::vector<double> y(n);
    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        for (int j = 0; j < i; ++j) sum += lu.L(i, j) * y[j];
        y[i] = bp[i] - sum;
    }

    // обратный ход U x = y
    std::vector<double> x(n);
    for (int i = n - 1; i >= 0; --i) {
        double sum = 0.0;
        for (int j = i + 1; j < n; ++j) sum += lu.U(i, j) * x[j];
        x[i] = (y[i] - sum) / lu.U(i, i);
    }
    return x;
}
```

Результат:

=== Решение системы (x) ===

```
2.00000000
4.00000000
2.00000000
3.00000000
```

=== Вектор перестановок P ===

```
1
3
2
0
```

=== Матрица L ===

1.00000000	0.00000000	0.00000000	0.00000000
0.66666667	1.00000000	0.00000000	0.00000000
-0.33333333	-0.50000000	1.00000000	0.00000000
-0.33333333	-0.50000000	1.00000000	1.00000000

=== Матрица U ===

-3.00000000	-5.00000000	14.00000000	13.00000000
0.00000000	-0.66666667	-4.33333333	1.33333333
0.00000000	0.00000000	0.50000000	3.00000000
0.00000000	0.00000000	0.00000000	8.00000000

=== Определитель det(A) = 8.00000000 ===

=== Обратная матрица A⁻¹ ===

-11.50000000	-2.00000000	42.50000000	18.00000000
5.12500000	1.00000000	-18.12500000	-8.00000000
-0.75000000	0.00000000	2.75000000	1.00000000
0.12500000	0.00000000	-0.12500000	0.00000000

=== Проверка: ||A x - b|| ===

0.00000000 (должно быть близко к 0)

Метод прогонки

Метод прогонки является частным случаем метода Гаусса. Он применяется для решения СЛАУ с трехдиагональными матрицами. Рассмотрим СЛАУ:

$$\left\{ \begin{array}{l} b_1 x_1 + c_1 x_2 = d_1 \\ a_2 x_1 + b_2 x_2 + c_2 x_3 = d_2 \\ a_3 x_2 + b_3 x_3 + c_3 x_4 = d_3 \\ \dots \\ a_{n-1} x_{n-2} + b_{n-1} x_{n-1} + c_{n-1} x_n = d_{n-1} \\ a_n x_{n-1} + b_n x_n = d_n \end{array} \right.$$

При этом будем полагать, что $a_1 = 0$ и $c_n = 0$. Решение системы можно искать в виде:

$$x_i = P_i x_{i+1} + Q_i, \quad i = \overline{1, n}$$

Здесь P_i и Q_i – прогоночные коэффициенты, определяемые по формулам:

$$P_1 = \frac{-c_1}{b_1}; \quad Q_1 = \frac{d_1}{b_1}$$

$$P_i = \frac{-c_i}{b_i + a_i P_{i-1}}; \quad Q_i = \frac{d_i - a_i Q_{i-1}}{b_i + a_i P_{i-1}}$$

После того как будут найдены прогоночные коэффициенты (прямой ход), можно вычислить значения неизвестных путем обратной подстановки (обратный ход):

$$x_n = P_n x_{n+1} + Q_n = 0 \cdot x_{n+1} + Q_n = Q_n$$

$$x_{n-1} = P_{n-1} x_n + Q_{n-1}$$

$$x_{n-2} = P_{n-2} x_{n-1} + Q_{n-2}$$

... ..

$$x_1 = P_1 x_2 + Q_1$$

Достаточным условием корректности метода прогонки и устойчивости его к погрешностям вычислений является условие преобладания диагональных коэффициентов:

$$|b_i| \geq |a_i| + |c_i|$$

Входные данные:

$$\begin{cases} -11 \cdot x_1 - 9 \cdot x_2 = -122 \\ 5 \cdot x_1 - 15 \cdot x_2 - 2 \cdot x_3 = -48 \\ -8 \cdot x_2 + 11 \cdot x_3 - 3 \cdot x_4 = -14 \\ 6 \cdot x_3 - 15 \cdot x_4 + 4 \cdot x_5 = -50 \\ 3 \cdot x_4 + 6 \cdot x_5 = 42 \end{cases}$$

Метод решения СЛАУ с трёхдиагональной матрицей:

```
std::vector<double> tridiagSolve(const std::vector<double>& a,
                                const std::vector<double>& b,
                                const std::vector<double>& c,
                                const std::vector<double>& d,
                                std::ostream& log) {
    const size_t n = b.size();
    if (n == 0) {
        return {};
    }

    if (d.size() != n) {
        throw std::invalid_argument("Размер d должен совпадать с размером b");
    }
    if (a.size() != (n > 0 ? n - 1 : 0)) {
        throw std::invalid_argument("Размер a должен быть n-1");
    }
    if (c.size() != (n > 0 ? n - 1 : 0)) {
        throw std::invalid_argument("Размер c должен быть n-1");
    }

    std::vector<double> p(n, 0.0);
    std::vector<double> q(n, 0.0);
    std::vector<double> x(n, 0.0);

    log << "=== Метод прогонки ===\n";
    log << "Решение ищется в виде x[i] = p[i] * x[i+1] + q[i]\n\n";

    if (n == 1) {
        if (std::fabs(b[0]) < EPS) {
            throw std::runtime_error("Нулевой элемент на главной диагонали: b[0] = 0");
        }

        p[0] = 0.0;
        q[0] = d[0] / b[0];
        x[0] = q[0];

        log << "n = 1:\n";
        log << "p[0] = 0\n";
        log << "q[0] = d[0] / b[0] = " << q[0] << "\n";
        log << "x[0] = q[0] = " << x[0] << "\n";

        return x;
    }

    log << "Прямой ход:\n";

    if (std::fabs(b[0]) < EPS) {
        throw std::runtime_error("Нулевой знаменатель на первом шаге: b[0] = 0");
    }

    p[0] = -c[0] / b[0];
    q[0] = d[0] / b[0];
```



```

log << "i = 0: "
  << "p[0] = " << p[0]
  << ", q[0] = " << q[0] << "\n";

for (size_t i = 1; i < n; ++i) {
  double denom = b[i] + a[i - 1] * p[i - 1];
  if (std::fabs(denom) < EPS) {
    throw std::runtime_error("Нулевой знаменатель на шаге i = " +
std::to_string(i));
  }

  if (i < n - 1) {
    p[i] = -c[i] / denom;
  } else {
    p[i] = 0.0; // p[n-1] = 0
  }

  q[i] = (d[i] - a[i - 1] * q[i - 1]) / denom;

  log << "i = " << i << ": "
    << "denom = b[" << i << "] + a[" << (i - 1) << "] * p[" << (i - 1) << "] = "
<< denom
    << ", p[" << i << "] = " << p[i]
    << ", q[" << i << "] = " << q[i] << "\n";
  }

  log << "\nОбратный ход:\n";

  x[n - 1] = q[n - 1];
  log << "x[" << (n - 1) << "] = q[" << (n - 1) << "] = " << x[n - 1] << "\n";

  for (int i = static_cast<int>(n) - 2; i >= 0; --i) {
    x[i] = p[i] * x[i + 1] + q[i];
    log << "x[" << i << "] = p[" << i << "] * x[" << (i + 1) << "] + q[" << i << "] = "
" << x[i] << "\n";
  }

  log << "\nМетод прогонки завершён успешно\n";
  return x;
}

```

Результат:

=== Решение системы x ===

```

7.00000000
5.00000000
4.00000000
6.00000000
4.00000000

```

=== Проверка ||A x - b|| ===

0.00000000 (должно быть близко к 0)

Итерационные методы решения СЛАУ

Рассмотрим СЛАУ

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \dots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

с невырожденной матрицей.

Приведем СЛАУ к эквивалентному виду

$$\begin{cases} x_1 = \beta_1 + \alpha_{11}x_1 + \alpha_{12}x_2 + \dots + \alpha_{1n}x_n \\ x_2 = \beta_2 + \alpha_{21}x_1 + \alpha_{22}x_2 + \dots + \alpha_{2n}x_n \\ \dots \\ x_n = \beta_n + \alpha_{n1}x_1 + \alpha_{n2}x_2 + \dots + \alpha_{nn}x_n \end{cases}$$

или в векторно-матричной форме $x = \beta + \alpha x$.

Такое приведение может быть выполнено различными способами. Одним из наиболее распространенных является следующий. Разрешим систему относительно неизвестных при ненулевых диагональных элементах $a_{ii} \neq 0, i = \overline{1, n}$ (если какой-либо коэффициент на главной диагонали равен нулю, достаточно соответствующее уравнение поменять местами с любым другим уравнением). Получим следующие выражения для компонентов вектора β и матрицы α эквивалентной системы:

$$\beta_i = \frac{b_i}{a_{ii}}$$

$$\alpha_{ij} = -\frac{a_{ij}}{a_{ii}}, \quad i \neq j; \quad \alpha_{ij} = 0, \quad i = j$$

В качестве нулевого приближения $x^{(0)}$ вектора неизвестных примем вектор правых частей $x^{(0)} = \beta$. Тогда **метод простых итераций** примет вид:

$$\begin{cases} x^{(0)} = \beta \\ x^{(1)} = \beta + \alpha x^{(0)} \\ x^{(2)} = \beta + \alpha x^{(1)} \\ \dots \\ x^{(k)} = \beta + \alpha x^{(k-1)} \end{cases}$$

Достаточным условием сходимости является диагональное преобладание матрицы A по строкам или по столбцам:

$$|a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}|$$

Критерием окончания итерационного процесса может служить неравенство $\|x^{(k)} - x^{(k-1)}\| \leq \varepsilon$.

Метод простых итераций довольно медленно сходится. Для его ускорения существует **метод Зейделя**, заключающийся в том, что при вычислении компонента x_i^{k+1} вектора неизвестных на $(k+1)$ -й итерации используются $x_1^{k+1}, x_2^{k+1}, \dots, x_{i-1}^{k+1}$, уже вычисленные на $(k+1)$ -й итерации. Тогда метод Зейделя для известного вектора на k -ой итерации имеет вид:

$$\begin{cases} x_1^{k+1} = \beta_1 + \alpha_{11}x_1^k + \alpha_{12}x_2^k + \dots + \alpha_{1n}x_n^k \\ x_2^{k+1} = \beta_2 + \alpha_{21}x_1^k + \alpha_{22}x_2^k + \dots + \alpha_{2n}x_n^k \\ x_3^{k+1} = \beta_3 + \alpha_{31}x_1^k + \alpha_{32}x_2^k + \dots + \alpha_{3n}x_n^k \\ \dots \\ x_n^{k+1} = \beta_n + \alpha_{n1}x_1^{k+1} + \alpha_{n2}x_2^{k+1} + \dots + \alpha_{nn-1}x_{n-1}^{k+1} + \alpha_{nn}x_n^k \end{cases}$$

Входные данные:
$$\begin{cases} 19 \cdot x_1 - 4 \cdot x_2 - 9 \cdot x_3 - x_4 = 100 \\ -2 \cdot x_1 + 20 \cdot x_2 - 2 \cdot x_3 - 7 \cdot x_4 = -5 \\ 6 \cdot x_1 - 5 \cdot x_2 - 25 \cdot x_3 + 9 \cdot x_4 = 34 \\ -3 \cdot x_2 - 9 \cdot x_3 + 12 \cdot x_4 = 69 \end{cases}$$

Метод простых итераций

```
std::pair<std::vector<double>, int> simpleIteration(
    const Matrix& A, const std::vector<double>& b, double eps,
    std::ostream& log) {

    size_t n = A.size();
    if (b.size() != n) {
        throw std::invalid_argument("simpleIteration: размер b не совпадает с размером
A");
    }

    Matrix alpha = buildAlpha(A);
    std::vector<double> beta = buildBeta(A, b);
    std::vector<double> x = beta; // x^(0) = beta
    std::vector<double> x_new(n);

    double alphaNorm = matrixNormC(alpha);

    log << "=== Метод простых итераций (Якоби) ===\n";
    log << "Начальное приближение: x^(0) = beta\n";
    log << "||alpha||_c = " << alphaNorm << "\n";
    log << "Точность eps = " << eps << ", max итераций = " << MAX_ITER << '\n';

    if (alphaNorm < 1.0) {
        log << "Достаточное условие сходимости ||alpha|| < 1 выполнено\n";
    } else {
        log << "ВНИМАНИЕ: ||alpha|| >= 1, достаточное условие сходимости не выпол-
нено\n";
    }
}
```

```

        log << "Используется эвристический критерий остановки:  $\|x^{(k)} - x^{(k-1)}\|_c < \text{eps}$ \n";
    }

    int iter = 0;
    while (iter < MAX_ITER) {
        //  $x^{(k+1)} = \beta + \alpha * x^{(k)}$ 
        std::vector<double> alphaX = multiplyMatrixVector(alpha, x);
        x_new = addVectors(beta, alphaX);

        std::vector<double> diff = subtractVectors(x_new, x);
        double diffNorm = vectorNormC(diff);
        ++iter;

        if (alphaNorm < 1.0) {
            double est = alphaNorm / (1.0 - alphaNorm) * diffNorm;
            log << "Итерация " << iter
                << ":  $\|x^{(k)} - x^{(k-1)}\|_c =$ " << diffNorm
                << ", оценка погрешности = " << est << '\n';

            x = x_new;
            if (est <= eps) {
                log << "Сходимость достигнута по достаточному условию\n";
                return {x, iter};
            }
        } else {
            log << "Итерация " << iter
                << ":  $\|x^{(k)} - x^{(k-1)}\|_c =$ " << diffNorm << '\n';

            x = x_new;
            if (diffNorm <= eps) {
                log << "Сходимость достигнута по эвристическому критерию\n";
                return {x, iter};
            }
        }
    }

    log << "ВНИМАНИЕ: метод простых итераций не сошёлся за " << MAX_ITER << " итераций\n";
    return {x, MAX_ITER};
}

```

Метод Зейделя

```

std::pair<std::vector<double>, int> gaussSeidel(
    const Matrix& A, const std::vector<double>& b, double eps,
    std::ostream& log) {

    size_t n = A.size();
    if (b.size() != n) {
        throw std::invalid_argument("gaussSeidel: размер b не совпадает с размером A");
    }

    Matrix alpha = buildAlpha(A);
    std::vector<double> beta = buildBeta(A, b);
    std::vector<double> x = beta;
    double alphaNorm = matrixNormC(alpha);

    log << "=== Метод Зейделя ===\n";
    log << "Начальное приближение:  $x^{(0)} = \beta$ \n";
    log << " $\|\alpha\|_c$  (для эквивалентной системы Якоби) = " << alphaNorm << "\n";
    log << "Точность eps = " << eps << ", max итераций = " << MAX_ITER << '\n';

    if (alphaNorm < 1.0) {
        log << "Достаточное условие сходимости выполнено\n";
    } else {
        log << "ВНИМАНИЕ:  $\|\alpha\| \geq 1$ , гарантии по достаточному условию нет\n";
    }
}

```

```

        log << "Используется эвристический критерий остановки:  $\|x^{(k)} - x^{(k-1)}\|_c < \epsilon$ \n";
    }

    int iter = 0;
    while (iter < MAX_ITER) {
        std::vector<double> x_old = x;

        //  $x_i^{(k+1)} = \beta_i + \sum_{j<i} \alpha_{ij} x_j^{(k+1)} + \sum_{j>i} \alpha_{ij} x_j^{(k)}$ 
        for (size_t i = 0; i < n; ++i) {
            double sum = beta[i];

            for (size_t j = 0; j < i; ++j) {
                sum += alpha(i, j) * x[j];
            }
            for (size_t j = i + 1; j < n; ++j) {
                sum += alpha(i, j) * x_old[j];
            }

            x[i] = sum;
        }

        std::vector<double> diff = subtractVectors(x, x_old);
        double diffNorm = vectorNormC(diff);

        ++iter;

        if (alphaNorm < 1.0) {
            double est = alphaNorm / (1.0 - alphaNorm) * diffNorm; // можно брать норму
            верхнетреуг. матрицы C, тогда будет круче
            log << "Итерация " << iter
                << ":  $\|x^{(k)} - x^{(k-1)}\|_c =$ " << diffNorm
                << ", оценка погрешности = " << est << '\n';

            if (est <= eps) {
                log << "Сходимость достигнута по достаточному условию\n";
                return {x, iter};
            }
        } else {
            log << "Итерация " << iter
                << ":  $\|x^{(k)} - x^{(k-1)}\|_c =$ " << diffNorm << '\n';

            if (diffNorm <= eps) {
                log << "Сходимость достигнута по эвристическому критерию\n";
                return {x, iter};
            }
        }
    }

    log << "ВНИМАНИЕ: метод Зейделя не сошёлся за " << MAX_ITER << " итераций\n";
    return {x, MAX_ITER};
}

```

Результат:

```

=== Метод простых итераций ===
Итераций: 48
Решение x:
8.00000019
3.99999985
2.99999973
9.00000032
 $\|Ax - b\|_1 = 0.00002972$ 

```

```

=== Метод Зейделя ===
Итераций: 21
Решение x:
7.99999960
3.99999972

```

2.99999977
 8.99999975
 $\|A x - b\|_1 = 0.00000939$

=== Анализ ===
 Метод Зейделя сошёлся быстрее

Метод вращений

Метод вращений Якоби применим только для симметрических матриц ($A^T = A$) и решает полную проблему собственных значений и собственных векторов таких матриц. Он основан на отыскании с помощью итерационных процедур матрицы U в преобразовании подобия $\Lambda = U^{-1}AU$, а поскольку для симметрических матриц A матрица преобразования подобия U является ортогональной ($U^{-1} = U^T$), то $\Lambda = U^T A U$, где Λ – диагональная матрица с собственными значениями на главной диагонали.

Пусть дана симметрическая матрица A . Требуется для нее вычислить с точностью ε все собственные значения и соответствующие им собственные векторы. Алгоритм метода вращений:

Пусть известна матрица $A^{(k)}$ на k -й итерации, при этом для $k=0$: $A^{(0)} = A$.

1. Выбирается максимальный по модулю недиагональный элемент $a_{ij}^{(k)}$ матрицы $A^{(k)}$ ($|a_{ij}^{(k)}| = \max_{l < m} |a_{lm}^{(k)}|$).
2. Ставится задача найти такую ортогональную матрицу $U^{(k)}$, чтобы в результате преобразования подобия $A^{(k+1)} = U^{(k)T} A^{(k)} U^{(k)}$ произошло обнуление элемента $a_{ij}^{(k+1)}$ матрицы $A^{(k+1)}$. В качестве ортогональной матрицы выбирается матрица вращения, имеющая следующий вид:

$$U^k = \begin{pmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & 1 & & & & \\ & & & \ddots & & & \\ & & & & 1 & & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix} \begin{matrix} \\ \\ \\ \\ \\ \\ \\ \end{matrix},$$

$$U^k = \begin{pmatrix} \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \cos \varphi^{(k)} & & -\sin \varphi^{(k)} & \vdots & \vdots & \vdots \\ \vdots & & 1 & & \vdots & \vdots & \vdots \\ \vdots & & & \ddots & & \vdots & \vdots \\ \vdots & & & & 1 & & \vdots \\ \vdots & \sin \varphi^{(k)} & & \cos \varphi^{(k)} & & \vdots & \vdots \\ \vdots & & & & & 1 & \vdots \\ 0 & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & 1 \end{pmatrix} \begin{matrix} \\ i \\ \\ j \\ \\ \end{matrix}$$

Угол вращения $\varphi^{(k)}$ определяется из условия $a_{ij}^{(k+1)} = 0$:

$$\varphi^{(k)} = \frac{1}{2} \operatorname{arctg} \frac{2a_{ij}^{(k)}}{a_{ii}^{(k)} - a_{jj}^{(k)}}$$

причем если $a_{ii}^{(k)} = a_{jj}^{(k)}$, то $\varphi^{(k)} = \frac{\pi}{4}$.

3. Строится матрица $A^{(k+1)}$

$$A^{(k+1)} = U^{(k)T} A^{(k)} U^{(k)}$$

В качестве критерия окончания итерационного процесса используется условие малости суммы квадратов внедиагональных элементов:

$$t(A^{(k+1)}) = \left(\sum_{l,m;l < m} (a_{lm}^{(k+1)})^2 \right)^{\frac{1}{2}}$$

Координатными столбцами собственных векторов матрицы A в единичном базисе будут столбцы матрицы $U = U^{(0)} U^{(1)} \dots U^{(k)}$.

Входные данные: $\begin{pmatrix} -7 & 4 & 5 \\ 4 & -6 & -9 \\ 5 & -9 & -8 \end{pmatrix}$

Метод вращений:

```
EigenResult jacobiEigen(const Matrix& A_orig, double eps, std::ostream& log) {
    size_t n = A_orig.size();

    // Проверка симметричности
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = i + 1; j < n; ++j) {
            if (std::fabs(A_orig(i, j) - A_orig(j, i)) > EPS_ZERO) {
                throw std::runtime_error("Метод вращений Якоби применим только к симметрическим матрицам");
            }
        }
    }

    Matrix A = A_orig;
    Matrix U(n); // накопленная матрица собственных векторов
    for (size_t i = 0; i < n; ++i) { U(i, i) = 1.0; }

    log << "=== Метод вращений (Якоби для собственных значений) ===\n";
    log << "Точность eps = " << eps << ", max итераций = " << MAX_ITER << "\n";

    std::vector<std::pair<double, int>> error_history;

    int iter = 0;
    while (iter < MAX_ITER) {
        // Критерий остановки:
        // t(A^(k)) = sqrt(sum_{l<m} (a_lm^(k))^2)
        double t = 0.0;
        for (size_t l = 0; l < n; ++l) {
            for (size_t m = l + 1; m < n; ++m) {
                t += A(l, m) * A(l, m);
            }
        }
        t = std::sqrt(t);
    }
```

```

log << "Итерация " << iter << ": t(A) = " << t << "\n";

if (t < eps) {
    log << "Сошлось за " << iter << " итераций\n";
    break;
}

// Выбор максимального по модулю внедиагонального элемента a_lm
size_t l = 0, m = 1;
double max_val = std::fabs(A(0, 1));
for (size_t i = 0; i < n; ++i) {
    for (size_t j = i + 1; j < n; ++j) {
        double val = std::fabs(A(i, j));
        if (val > max_val) {
            max_val = val;
            l = i;
            m = j;
        }
    }
}

const double all = A(l, l);
const double amm = A(m, m);
const double alm = A(l, m);

// Угол вращения
double phi;
if (std::fabs(all - amm) < EPS_ZERO) {
    phi = M_PI / 4.0;
} else {
    phi = 0.5 * std::atan((2.0 * alm) / (all - amm));
}

const double c = std::cos(phi);
const double s = std::sin(phi);

log << "  max |a_lm| = " << max_val
    << " при l = " << l << ", m = " << m << "\n";
log << "  phi = " << phi << "\n";

// Обновление матрицы A
for (size_t k = 0; k < n; ++k) {
    if (k == l || k == m) continue;

    const double ak_l = A(k, l);
    const double ak_m = A(k, m);

    const double new_kl = c * ak_l + s * ak_m;
    const double new_km = -s * ak_l + c * ak_m;

    A(k, l) = new_kl;
    A(l, k) = new_kl;
    A(k, m) = new_km;
    A(m, k) = new_km;
}

A(l, l) = c * c * all + 2.0 * c * s * alm + s * s * amm;
A(m, m) = s * s * all - 2.0 * c * s * alm + c * c * amm;
A(l, m) = 0.0;
A(m, l) = 0.0;

// Обновление матрицы собственных векторов
for (size_t k = 0; k < n; ++k) {
    const double uk_l = U(k, l);
    const double uk_m = U(k, m);

    U(k, l) = c * uk_l + s * uk_m;
    U(k, m) = -s * uk_l + c * uk_m;
}

```



```

    }

    ++iter;
}

if (iter == MAX_ITER) {
    log << "ВНИМАНИЕ: не сошлось за " << MAX_ITER << " итераций!\n";
}

// Собственные значения – диагональ A
std::vector<double> eigenvalues(n);
for (size_t i = 0; i < n; ++i) {
    eigenvalues[i] = A(i, i);
}

return {eigenvalues, U};
}

```

Результат:

```

=== Собственные значения ===
-3.71064850
2.07377213
-19.36312363

=== Собственные векторы (столбцы) ===
0.88666192 -0.05184815 -0.45950235
0.34682369 0.73180676 0.58666191
0.30584959 -0.67953707 0.66684736

=== Проверка реконструкции ===
||A - U * Lambda * U^T||_1 = 0.00000012

=== Проверка по определению: ||A u_i - lambda_i u_i||_1 ===
i = 0: 0.00000000
i = 1: 0.00000007
i = 2: 0.00000006

```

QR-алгоритм нахождения собственных значений матриц

В основе QR -алгоритма лежит представление матрицы в виде $A = QR$, где Q – ортогональная матрица ($Q^{-1} = Q^T$), а R – верхняя треугольная. Такое разложение существует для любой квадратной матрицы. Одним из возможных подходов к построению QR -разложения является использование преобразования Хаусхолдера, позволяющего обратить в нуль группу поддиагональных элементов столбца матрицы.

Преобразование Хаусхолдера осуществляется с использованием матрицы Хаусхолдера, имеющей следующий вид:

$$H = E - \frac{2}{v^T v} v v^T, \text{ где } v - \text{произвольный ненулевой столбец.}$$

Рассмотрим случай, когда необходимо обратить в нуль все элементы какого-либо вектора кроме первого, т.е. построить матрицу Хаусхолдера такую, что

$$\tilde{b} = Hb, b = (b_1, b_2, \dots, b_n)^T, \tilde{b} = (\tilde{b}_1, 0, \dots, 0)^T$$

Тогда вектор v определится следующим образом:

$$v = b + \text{sign}(b_1) \|b\|_2 e_1$$

Применяя описанную процедуру с целью обнуления поддиагональных элементов каждого из столбцов исходной матрицы, можно за фиксированное число шагов получить ее QR – разложение.

Процедура QR-разложения многократно используется в QR-алгоритме вычисления собственных значений. Строится следующий итерационный процесс:

$$A^{(0)} = A,$$

$$A^{(0)} = Q^{(0)} R^{(0)} \text{ – производится QR-разложение,}$$

$$A^{(1)} = R^{(0)} Q^{(0)} \text{ – производится перемножение матриц,}$$

.....

$$A^{(k)} = Q^{(k)} R^{(k)} \text{ – разложение}$$

$$A^{(k+1)} = R^{(k)} Q^{(k)} \text{ – перемножение}$$

Таким образом, каждая итерация реализуется в два этапа. На первом этапе осуществляется разложение матрицы $A^{(k)}$ в произведение ортогональной $Q^{(k)}$ и верхней треугольной $R^{(k)}$ матриц, а на втором – полученные матрицы перемножаются в обратном порядке.

При отсутствии у матрицы кратных собственных значений последовательность $A^{(k)}$ сходится к верхней треугольной матрице (в случае, когда все собственные значения вещественны) или к верхней квазитреугольной матрице (если имеются комплексносопряженные пары собственных значений).

Таким образом, каждому вещественному собственному значению будет соответствовать столбец со стремящимися к нулю поддиагональными элементами и в качестве критерия сходимости итерационного процесса для таких собственных значений можно использовать следующее неравенство:

$(\sum_{l=m+1}^n (a_{lm}^{(k)})^2)^{\frac{1}{2}} \leq \varepsilon$. При этом соответствующее собственное значение принимается равным диагональному элементу данного столбца.

Входные данные: $\begin{pmatrix} 3 & -7 & -1 \\ -9 & -8 & 7 \\ 5 & 2 & 2 \end{pmatrix}$

Матрица Хаусхолдера:

```
// Формируем отражение Хаусхолдера H = I - 2 vv^T/(v^T v),
// где v имеет размер n, но ненулевые компоненты только начиная с offset.
Matrix buildHouseholderFromTail(const std::vector<double>& x, size_t offset) {
    const size_t n = x.size();
    Matrix H = Matrix::identityMatrix(n);

    double normx = 0.0;
    for (size_t i = offset; i < n; ++i) {
        normx += x[i] * x[i];
    }
    normx = std::sqrt(normx);

    if (normx < EPS_ZERO) {
        return H;
    }

    std::vector<double> v = x;
    v[offset] += signNonZero(x[offset]) * normx;

    double vtv = 0.0;
    for (size_t i = offset; i < n; ++i) {
        vtv += v[i] * v[i];
    }

    if (vtv < EPS_ZERO) {
        return H;
    }

    for (size_t i = offset; i < n; ++i) {
        for (size_t j = offset; j < n; ++j) {
            H(i, j) -= 2.0 * v[i] * v[j] / vtv;
        }
    }

    return H;
}
```

QR-разложение:

```
QRDecomposition qrDecomposeHouseholder(const Matrix& A_orig, std::ostream& log) {
    const size_t n = A_orig.size();

    Matrix R = A_orig;
    Matrix Q = Matrix::identityMatrix(n);

    log << "=== QR-разложение (Хаусхолдер) ===\n";

    for (size_t k = 0; k + 1 < n; ++k) {
        std::vector<double> x(n, 0.0);
        for (size_t i = k; i < n; ++i) {
            x[i] = R(i, k);
        }

        Matrix H = buildHouseholderFromTail(x, k);

        R = multiply(H, R);
        Q = multiply(Q, H); // H^T = H
    }
}
```

```

        log << "Шаг k = " << k << ": обнуление поддиагональных элементов столбца " << k
    << "\n";
    }

    return {Q, R};
}

```

QR-алгоритм:

```

QREigenResult qrAlgorithm(const Matrix& A_orig, double eps, std::ostream& log) {
    const size_t n = A_orig.size();
    if (eps <= 0.0) {
        throw std::invalid_argument("qrAlgorithm: eps must be positive");
    }

    log << "=== QR-алгоритм ===\n";

    Matrix A = A_orig;

    std::vector<std::complex<double>> prevComplex(n, std::complex<double>(0.0, 0.0));
    std::vector<bool> hasPrevComplex(n, false);

    int iter = 0;
    while (iter < MAX_ITER) {
        log << "\nИтерация " << iter
            << ": ||strict lower||_2 = " << strictLowerNorm(A) << "\n";

        // 1) Сначала попробуем выделить завершившиеся блоки
        bool all_done = true;
        size_t i = 0;
        while (i < n) {
            // последний 1x1 блок
            if (i == n - 1) {
                ++i;
                continue;
            }

            // вещественное собственное значение, критерий:
            // столбец i имеет малую норму под диагональю
            double col_norm = columnSubdiagNorm(A, i);
            if (col_norm <= eps) {
                ++i;
                continue;
            }

            // Иначе рассматриваем кандидат на 2x2 блок
            auto cur = eigenvalues2x2(
                A(i, i), A(i, i + 1),
                A(i + 1, i), A(i + 1, i + 1)
            );

            if (iter > 0 && hasPrevComplex[i]) {
                // Учитываем возможную перестановку корней между итерациями
                double directDiff =
                    std::abs(cur.first - prevComplex[i]) +
                    std::abs(cur.second - prevComplex[i + 1]);

                double swappedDiff =
                    std::abs(cur.first - prevComplex[i + 1]) +
                    std::abs(cur.second - prevComplex[i]);

                if (std::min(directDiff, swappedDiff) <= eps) {
                    i += 2;
                    continue;
                }
            }

            prevComplex[i] = cur.first;
            prevComplex[i + 1] = cur.second;
        }
    }
}

```

```

        hasPrevComplex[i] = true;

        all_done = false;
        i += 2;
    }

    if (all_done) {
        log << "Критерий остановки выполнен\n";
        break;
    }

    // 2) QR-разложение и шаг  $A_{k+1} = R_k Q_k$ 
    auto qr = qrDecomposeHouseholder(A, log);
    A = multiply(qr.R, qr.Q);

    ++iter;
}

if (iter == MAX_ITER) {
    log << "ВНИМАНИЕ: достигнут предел по числу итераций\n";
}

// Извлечение собственных значений из почти квазитреугольной матрицы
std::vector<std::complex<double>> eigenvalues;
size_t i = 0;
while (i < n) {
    if (i == n - 1 || columnSubdiagNorm(A, i) <= eps) {
        eigenvalues.emplace_back(A(i, i), 0.0);
        ++i;
    } else {
        auto ev = eigenvalues2x2(
            A(i, i), A(i, i + 1),
            A(i + 1, i), A(i + 1, i + 1)
        );
        eigenvalues.push_back(ev.first);
        eigenvalues.push_back(ev.second);
        i += 2;
    }
}

return {eigenvalues, A, iter};
}

```

Результат:

=== Собственные значения ===

-13.50104225

5.25052113 + 2.86479382i

5.25052113 - 2.86479382i

=== Финальная матрица A_k ===

-13.50104225 1.87123950 -3.96163303

0.00000017 3.45719521 3.55102416

-0.00000007 -3.21683574 7.04384704

=== Число итераций ===

23

Лабораторная работа №2

О программе

Программа написана на языке C++ с использованием стандарта C++23. Решение реализовано модульно: методы решения нелинейных уравнений и систем нелинейных уравнений вынесены в отдельные классы и исходные файлы. Для вспомогательных операций с матрицами используются модули, реализованные в первой лабораторной работе.

Инструкция к запуску

Проект собирается с помощью CMake. Все входные данные хранятся в директории input в формате txt. При запуске программы последовательно выполняются все задания лабораторной работы, результаты сохраняются в директорию output. Для построения графиков дополнительно используются Python-скрипты из директории plots.

Пример для выполнения всех заданий:

```
cmake --build build
./main
```

Решение нелинейных уравнений

Метод простых итераций:

Пусть требуется решить трансцендентное уравнение вида $f(x) = 0$. Предположим, что корень уравнения отделен и находится на отрезке $[a, b]$. Кроме того, функция удовлетворяет некоторым дополнительным условиям:

- на концах отрезка функция имеет разные знаки;
- $\forall x \in [a, b]: f'(x) \neq 0$;
- первая и вторая производные имеют постоянные знаки.

Для того чтобы построить итерационный процесс, согласно методу простой итерации уравнение $f(x) = 0$ заменяется эквивалентным уравнением: $x = \varphi(x)$, причем $\varphi(x)$ – непрерывная функция. Выберем некоторое нулевое приближение x^0 , а затем организуем итерационный процесс по схеме:

$$x^{(k+1)} = \varphi(x^{(k)})$$

Условие $|\varphi'(x)| \leq q < 1, \forall x \in [a, b]$ является достаточным условием сходимости итераций, если начальное приближение выбрано в некоторой окрестности корня.

Метод Ньютона:

Предположим, что на интервале $[a, b]$ требуется определить корень уравнения $f(x) = 0$. Для того чтобы построить итерационный процесс согласно методу Ньютона, непрерывная функция $f(x)$ на интервалах $x \in [a, b]$ должна удовлетворять условиям, аналогичным условиям в методе итераций:

Правило построения итерационной последовательности получается путем замены нелинейной функции $f(x)$ ее линейной моделью на основе формулы Тейлора. Выберем в окрестности решения уравнения две соседние точки, так что $x^{(k+1)} = x^{(k)} + \varepsilon$, и запишем разложение функции в ряд Тейлора:

$$f(x^{(k+1)}) = f(x^{(k)}) + f'(x^{(k)})(x^{(k+1)} - x^{(k)}) + \frac{1}{2}f''(x^{(k)})(x^{(k+1)} - x^{(k)})^2 + \dots$$

Учитывая, что $f(x_{k+1}) \approx 0$ и оставляя только линейную часть разложения ряда, запишем соотношение метода Ньютона:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Используя условие сходимости метода простых итераций, получим достаточное условие сходимости метода Ньютона в форме

$$|f(x) \cdot f''(x)| < (f'(x))^2, x \in [a, b]$$

Для выбора начального приближения $x^{(0)}$ используется теорема, которая гласит, что в качестве начального приближения нужно выбрать тот конец интервала, где знак функции совпадает со знаком второй производной:

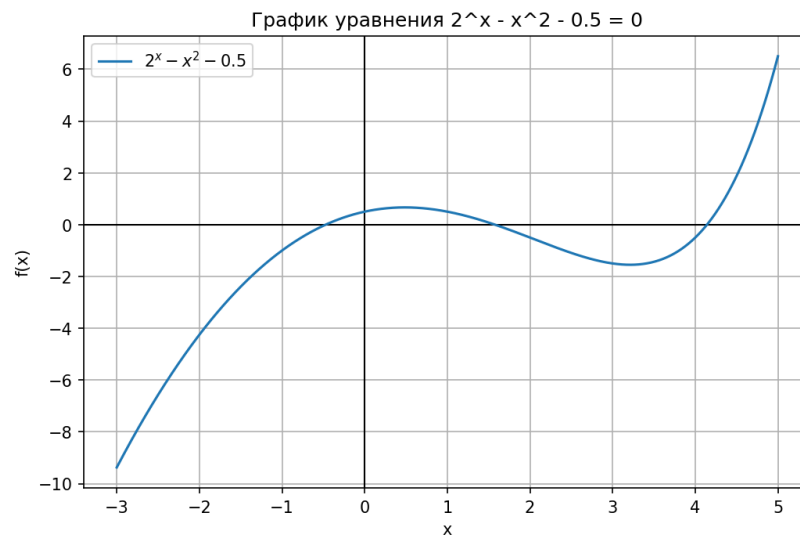
$$x^{(0)} = \begin{cases} a, & \text{если } f(a) \cdot f''(a) > 0 \\ b, & \text{если } f(b) \cdot f''(b) > 0 \end{cases}$$

Для завершения итерационного процесса используется правило:

$$|x^{(k+1)} - x^{(k)}| < \varepsilon$$

Уравнение: $2^x - x^2 - 0.5 = 0$

График:



Метод простых итераций:

```
RootResult solveSimpleIteration(double a, double b, double eps, std::ostream& log) {
    log << "---- Метод простой итерации ----\n";

    PhiType type = choosePhi(a, b);

    log << "Выбранная phi: ";
    if (type == PhiType::NegativeRoot) {
        log << "-sqrt(2^x - 0.5)\n";
    } else if (type == PhiType::PositiveMiddleRoot) {
        log << "sqrt(2^x - 0.5)\n";
    } else {
        log << "log2(x^2 + 0.5)\n";
    }

    double q = 0.0;
    bool q_ok = estimateQ(a, b, q, type);
    bool invariant = mapsIntervalToItself(a, b, type);

    if (!q_ok) {
        log << "phi или phi' не определены на всём интервале\n";
        return {std::numeric_limits<double>::quiet_NaN(), 0};
    }

    log << "Оценка q = " << q << "\n";
    log << "phi([a,b]) subset [a,b]: " << (invariant ? "да" : "нет") << "\n";

    if (!invariant) {
        log << "ВНИМАНИЕ: phi не отображает интервал в себя\n";
        return {std::numeric_limits<double>::quiet_NaN(), 0};
    }

    if (q >= 1.0) {
        log << "ВНИМАНИЕ: условие сходимости |phi'(x)| < 1 не выполнено\n";
        return {std::numeric_limits<double>::quiet_NaN(), 0};
    }

    double x = (a + b) / 2.0;
    log << "Начальное приближение x0 = " << x << "\n";

    int iter = 0;
    while (iter < MAX_ITER) {
        double x_new;
```



```

    if (!phiValue(x, type, x_new)) {
        log << "Итерация вышла в область, где phi не определена\n";
        return {std::numeric_limits<double>::quiet_NaN(), iter};
    }

    double diff = std::fabs(x_new - x);
    double est = q / (1.0 - q) * diff;

    log << "k=" << iter
        << " x=" << x_new
        << " f(x)=" << f(x_new)
        << " |dx|=" << diff
        << " est=" << est << "\n";

    if (est < eps) {
        return {x_new, iter + 1};
    }

    x = x_new;
    ++iter;
}

log << "He сошлось\n";
return {x, MAX_ITER};
}

```

Метод Ньютона:

```

RootResult solveNewton(double a, double b, double eps, std::ostream& log) {
    log << "---- Метод Ньютона ----\n";

    double x;

    if (f(a) * ddf(a) > 0) x = a;
    else if (f(b) * ddf(b) > 0) x = b;
    else {
        log << "He удалось выбрать начальное приближение строго по условию\n";
        x = (a + b) / 2.0;
    }

    log << "Начальное приближение x0 = " << x << "\n";

    int iter = 0;

    while (iter < MAX_ITER) {
        double fx = f(x);
        double dfx = df(x);

        if (std::fabs(dfx) < EPS_ZERO)
            throw std::runtime_error("df(x) ~ 0");

        double x_new = x - fx / dfx;

        double diff = std::fabs(x_new - x);

        log << "k=" << iter
            << " x=" << x_new
            << " f(x)=" << f(x_new)
            << " |dx|=" << diff << "\n";

        if (diff < eps)
            return {x_new, iter + 1};

        x = x_new;
        iter++;
    }

    log << "He сошлось\n";
}

```

```

    return {x, MAX_ITER};
}

```

Результат:

```

=== Интервал [-0.70000000, -0.30000000] ===
Простая итерация:
root = -0.47073859, iter = 19
|f(root)| = 0.00000026
Ньютон:
root = -0.47073877, iter = 4
|f(root)| = 0.00000000

=== Интервал [1.00000000, 2.00000000] ===
Простая итерация:
root = 1.57382840, iter = 28
|f(root)| = 0.00000057
Ньютон:
root = 1.57382892, iter = 4
|f(root)| = 0.00000000

=== Интервал [4.00000000, 4.30000000] ===
Простая итерация:
root = 4.14231022, iter = 24
|f(root)| = 0.00000259
Ньютон:
root = 4.14230956, iter = 4
|f(root)| = 0.00000000

```

Решение систем нелинейных уравнений

Метод простых итераций:

Решение системы нелинейных уравнений вида

$$\begin{cases} f_1(x_1, x_2, x_3, \dots, x_n) = 0 \\ f_2(x_1, x_2, x_3, \dots, x_n) = 0 \\ \dots \\ f_n(x_1, x_2, x_3, \dots, x_n) = 0 \end{cases}$$

возможно, если все функции в системе непрерывны и дифференцируемы в окрестности решения.

Для использования метода итераций система уравнений записывается в эквивалентной форме:

$$\begin{cases} x_1 = \phi_1(x_1, x_2, x_3, \dots, x_n) \\ x_2 = \phi_2(x_1, x_2, x_3, \dots, x_n) \\ \dots \\ x_n = \phi_n(x_1, x_2, x_3, \dots, x_n) \end{cases},$$

где ϕ_i — итерирующие непрерывно дифференцируемые функции.

Тогда, если известно начальное приближение $X^{(0)}$, то можно построить алгоритм метода простых итераций:

$$\begin{cases} x_1^{(k+1)} = \phi_1(x_1^{(k)}, x_2^{(k)}, x_3^{(k)}, \dots, x_n^{(k)}) \\ x_2^{(k+1)} = \phi_2(x_1^{(k)}, x_2^{(k)}, x_3^{(k)}, \dots, x_n^{(k)}) \\ \dots \\ x_n^{(k+1)} = \phi_n(x_1^{(k)}, x_2^{(k)}, x_3^{(k)}, \dots, x_n^{(k)}) \end{cases}$$

Предложение формулировки достаточного условия сходимости для многомерного случая выглядит следующим образом. Метод простых итераций сходится к решению системы нелинейных уравнений, если какая-либо норма матрицы Якоби $J(X^{(k)})$, построенная по правым частям ϕ_i эквивалентной системы в замкнутой области G , меньше единицы на каждой итерации:

$$J(X^{(k)}) = \left[\frac{\partial \phi_i(X^{(k)})}{\partial x_j} \right]; \quad \|J(X^{(k)})\| \leq q < 1$$

Для практических расчетов чаще всего используют матричную норму, определенную в области решения G , которую обязательно проверяют в начальном приближении:

$$\max_{x \in G} \|\phi'(x)\| = \max_{x \in G} \left\{ \max_i \sum_{j=1}^n \left| \frac{\partial \phi_i(X)}{\partial x_j} \right| \right\}$$

В практических вычислениях в качестве условия окончания итераций обычно используется критерий

$$|X^{(k+1)} - X^{(k)}| < \varepsilon$$

Метод Ньютона:

Пусть дана система нелинейных уравнений. Все функции системы непрерывны на некотором интервале $[a, b]$ и дифференцируемы вплоть до вторых производных.

Итерационный процесс нахождения решения, который носит название метода Ньютона для систем, записывается в виде решения матричного уравнения:

$$X^{(k+1)} = X^{(k)} - J^{-1}(X^{(k)}) \cdot f(X^{(k)}),$$

где $J(X^{(k)})$ – матрица Якоби.

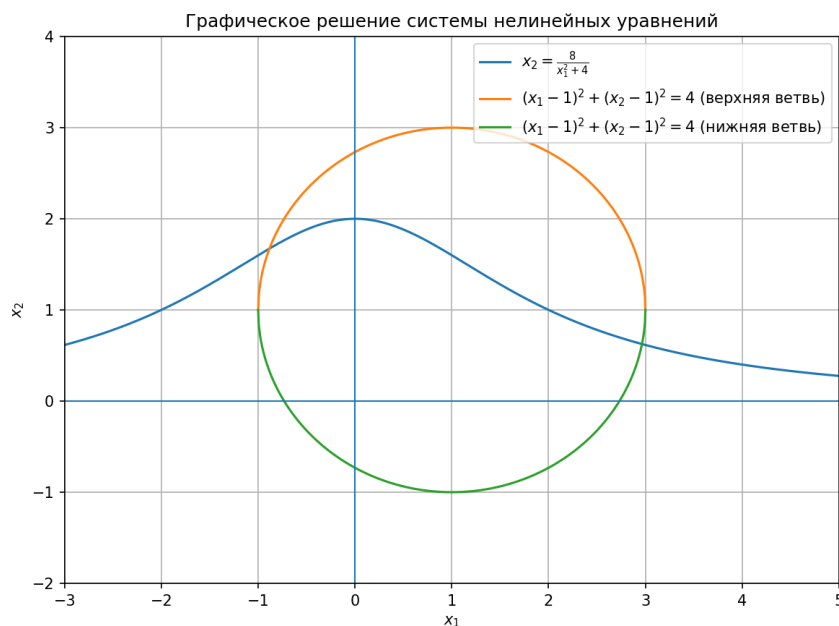
$$J(X^{(k)}) = \begin{vmatrix} \frac{\partial f_1(X^{(k)})}{\partial x_1} & \frac{\partial f_1(X^{(k)})}{\partial x_2} & \dots & \frac{\partial f_1(X^{(k)})}{\partial x_n} \\ \frac{\partial f_2(X^{(k)})}{\partial x_1} & \frac{\partial f_2(X^{(k)})}{\partial x_2} & \dots & \frac{\partial f_2(X^{(k)})}{\partial x_n} \\ \dots & \dots & \dots & \dots \\ \frac{\partial f_n(X^{(k)})}{\partial x_1} & \frac{\partial f_n(X^{(k)})}{\partial x_2} & \dots & \frac{\partial f_n(X^{(k)})}{\partial x_n} \end{vmatrix}$$

В практических вычислениях в качестве условия окончания итераций обычно используют критерий, выполняющийся для всех переменных системы:

$$|X^{(k+1)} - X^{(k)}| < \varepsilon$$

Система:
$$\begin{cases} (x_1^2 + a^2)x_2 - a^3 = 0, \\ (x_1 - a/2)^2 + (x_2 - a/2)^2 - a^2 = 0. \end{cases}, a = 2$$

График:



Метод простых итераций:

```
SystemResult solveSimpleIterationSystem(double a1, double b1,
                                         double a2, double b2,
                                         const std::vector<double>& x0,
                                         double eps,
                                         std::ostream& log) {
    if (x0.size() != 2) {
        throw std::invalid_argument("solveSimpleIterationSystem: x0 must have size 2");
    }
}
```

```

log << "---- Метод простой итерации для системы ----\n";

double q = estimateQ(a1, b1, a2, b2);
bool invariant = mapsRectangleToItself(a1, b1, a2, b2);

log << "Оценка q = " << q << "\n";
log << "phi(G) subset G: " << (invariant ? "да" : "нет") << "\n";

if (!std::isfinite(q)) {
    log << "phi или производные phi не определены на всей области\n";
    return {{std::numeric_limits<double>::quiet_NaN(),
              std::numeric_limits<double>::quiet_NaN()}}, 0};
}

if (!invariant) {
    log << "ВНИМАНИЕ: phi не отображает область в себя\n";
    return {{std::numeric_limits<double>::quiet_NaN(),
              std::numeric_limits<double>::quiet_NaN()}}, 0};
}

if (q >= 1.0) {
    log << "ВНИМАНИЕ: достаточное условие сходимости q < 1 не выполнено\n";
    return {{std::numeric_limits<double>::quiet_NaN(),
              std::numeric_limits<double>::quiet_NaN()}}, 0};
}

std::vector<double> x = x0;
log << "Начальное приближение x^(0) = (" << x[0] << ", " << x[1] << ") \n";

int iter = 0;
while (iter < MAX_ITER) {
    std::vector<double> x_new(2);
    x_new[0] = phi1(x[0], x[1]);
    x_new[1] = phi2(x[0], x[1]);

    auto diff = subtractVectors(x_new, x);
    double diffNorm = vectorNormC(diff);
    double est = q / (1.0 - q) * diffNorm;

    log << "k=" << iter
        << " x=(" << x_new[0] << ", " << x_new[1] << ")"
        << " F=(" << f1(x_new[0], x_new[1]) << ", " << f2(x_new[0], x_new[1]) << ")"
        << " ||dx||_c=" << diffNorm
        << " est=" << est << "\n";

    if (est < eps) {
        return {x_new, iter + 1};
    }

    x = x_new;
    ++iter;
}

log << "Не сошлось\n";
return {x, MAX_ITER};
}

```

Метод Ньютона:

```

SystemResult solveNewtonSystem(const std::vector<double>& x0,
                               double eps,
                               const VectorFunction& F,
                               const JacobianFunction& J,
                               std::ostream& log) {
    if (x0.empty()) {
        throw std::invalid_argument("solveNewtonSystem: x0 must be non-empty");
    }
}

```

```

std::vector<double> x = x0;
log << "---- Метод Ньютона для системы ----\n";
log << "Начальное приближение  $x^0 =$  (" ;
for (size_t i = 0; i < x.size(); ++i) {
    log << x[i] << (i + 1 == x.size() ? "" : ", ");
}
log << ")\n";

int iter = 0;
while (iter < MAX_ITER) {
    auto Fx = F(x);
    Matrix Jx = J(x);

    if (Fx.size() != x.size() || Jx.size() != x.size()) {
        throw std::runtime_error("solveNewtonSystem: inconsistent sizes of F or J");
    }

    std::vector<double> rhs = Fx;
    for (double& v : rhs) {
        v = -v;
    }

    auto lu = luDecompose(Jx, log);
    auto delta = solveLU(lu, rhs);
    auto x_new = addVectors(x, delta);

    double diffNorm = vectorNormC(delta);

    log << "k=" << iter << " x=(" ;
    for (size_t i = 0; i < x_new.size(); ++i) {
        log << x_new[i] << (i + 1 == x_new.size() ? "" : ", ");
    }
    log << ") F=(" ;
    auto Fnew = F(x_new);
    for (size_t i = 0; i < Fnew.size(); ++i) {
        log << Fnew[i] << (i + 1 == Fnew.size() ? "" : ", ");
    }
    log << ") ||delta||_c=" << diffNorm << "\n";

    if (diffNorm < eps) {
        return {x_new, iter + 1};
    }

    x = x_new;
    ++iter;
}

log << "Не сошлось\n";
return {x, MAX_ITER};
}

```

Результат:

=== Метод простой итерации ===

```

x1 = 2.96463133
x2 = 0.62553556
iter = 9
||F(x)||_c = 0.00000140

```

=== Метод Ньютона ===

```

x1 = 2.96463138
x2 = 0.62553565
iter = 3
||F(x)||_c = 0.00000000

```

Лабораторная работа №3

О программе

Программа написана на языке C++ с использованием стандарта C++23. Решение реализовано модульно: интерполяция, сплайны, метод наименьших квадратов, численное дифференцирование и численное интегрирование вынесены в отдельные классы и модули. Для некоторых вычислений используются матричные алгоритмы, реализованные в первой лабораторной работе.

Инструкция к запуску

Проект собирается с помощью CMake. Все входные данные хранятся в директории input в формате txt. При запуске программы последовательно выполняются все задания лабораторной работы, результаты сохраняются в директорию output. Для построения графиков используются Python-скрипты из директории plots.

Пример для выполнения всех заданий:

```
cmake --build build
./main
```

Интерполяция

Пусть на отрезке $[a, b]$ задано множество несовпадающих точек x_i (интерполяционных узлов), в которых известны значения функции $f_i = f(x_i)$. Приближающая функция $\varphi(x, a)$ такая, что выполняются равенства

$$\varphi(x_i, a_0, \dots, a_n) = f(x_i)$$

называется интерполяционной.

Наиболее часто в качестве приближающей функции используют многочлены степени n :

$$P_n(x) = \sum_{i=0}^n a_i x^i$$

Произвольный многочлен может быть записан в виде

$$L_n(x) = \sum_{i=0}^n f_i l_i(x)$$

Здесь $l_i(x)$ – многочлены степени n , так называемые лагранжевы многочлены влияния, которые удовлетворяют условию $l_i(x_j) = \begin{cases} 1, & \text{при } i = j \\ 0, & \text{при } i \neq j \end{cases}$ и, соответственно,

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{(x - x_j)}{(x_i - x_j)},$$

а интерполяционный многочлен запишется в виде

$$L_n(x) = \sum_{i=0}^n f_i \prod_{j=0, j \neq i}^n \frac{(x - x_j)}{(x_i - x_j)}$$

Интерполяционный многочлен, записанный в этой форме, называется **интерполяционным многочленом Лагранжа**.

Недостатком интерполяционного многочлена Лагранжа является необходимость полного пересчета всех коэффициентов в случае добавления дополнительных интерполяционных узлов. Чтобы избежать указанного недостатка используют интерполяционный многочлен в форме Ньютона.

Введем понятие разделенной разности. Разделенные разности нулевого порядка совпадают со значениями функции в узлах. Разделенные разности первого порядка обозначаются $f(x_i, x_j)$ и определяются через разделенные разности нулевого порядка:

$$f(x_i, x_j) = \frac{f_i - f_j}{x_i - x_j},$$

разделенные разности второго порядка определяются через разделенные разности первого порядка:

$$f(x_i, x_j, x_k) = \frac{f(x_i, x_j) - f(x_j, x_k)}{x_i - x_k}$$

Разделенная разность $n - k + 2$ определяется соотношениями

$$f(x_i, x_j, x_k, \dots, x_{n-1}, x_n) = \frac{f(x_i, x_j, x_k, \dots, x_{n-1}) - f(x_j, x_k, \dots, x_n)}{x_i - x_n}$$

Интерполяционный многочлен Ньютона может быть представлен в виде:

$$P_n(x) = f(x_0) + (x - x_0)f(x_1, x_0) + (x - x_0)(x - x_1)f(x_0, x_1, x_2) + \dots + (x - x_0)(x - x_1) \dots (x - x_{n-1})f(x_0, x_1, \dots, x_n)$$

Отметим, что при добавлении новых узлов первые члены многочлена Ньютона остаются неизменными.

Для повышения точности интерполяции в сумму могут быть добавлены новые члены, что требует подключения дополнительных интерполяционных узлов. При этом безразлично, в каком порядке подключаются новые узлы. Этим формула Ньютона выгодно отличается от формулы Лагранжа.

Входные данные: $y = \sin(x)$, а) $X_i = 0.1\pi, 0.2\pi, 0.3\pi, 0.4\pi$; б)

$$X_i = 0.1\pi, \frac{\pi}{6}, 0.3\pi, 0.4\pi; \quad X^* = \frac{\pi}{4}$$

Метод Лагранжа:

```
double lagrangeInterpolate(const std::vector<double>& x,
                           const std::vector<double>& y,
                           double xStar,
                           std::ostream& log) {
    validateNodes(x, y);

    const size_t n = x.size();
    double result = 0.0;

    log << "=== Интерполяционный многочлен Лагранжа ===\n";
    log << "L(x) = sum_{i=0}^{n-1} y[i] * l[i](x)\n\n";

    for (size_t i = 0; i < n; ++i) {
        double basis = 1.0;

        log << "i = " << i << ":\n";
        log << "  l[" << i << "]( " << xStar << ") = ";

        for (size_t j = 0; j < n; ++j) {
            if (j == i) continue;

            double denom = x[i] - x[j];
            if (std::fabs(denom) < EPS) {
                throw std::runtime_error("Нулевой знаменатель в Лагранже");
            }

            double factor = (xStar - x[j]) / denom;
            basis *= factor;

            log << "(" << xStar << " - " << x[j] << ")/(" << x[i] << " - " << x[j] <<
            "));";
            if (j + 1 < n) log << " * ";
        }
        log << " = " << basis << "\n";

        double term = y[i] * basis;
        result += term;

        log << "  term = y[" << i << "] * l[" << i << "] = "
            << y[i] << " * " << basis << " = " << term << "\n\n";
    }

    log << "L(" << xStar << ") = " << result << "\n\n";
    return result;
}
```

Метод Ньютона:

```
double newtonInterpolate(const std::vector<double>& x,
                        const std::vector<std::vector<double>>& dd,
                        double xStar,
                        std::ostream& log) {
    const size_t n = x.size();
    if (dd.size() != n) {
        throw std::invalid_argument("Некорректная таблица разделённых разностей");
    }

    log << "=== Интерполяционный многочлен Ньютона ===\n";

    double result = dd[0][0];
    log << "P(x) = f[x0]";
    for (size_t order = 1; order < n; ++order) {
        log << " + f[x0]";
        for (size_t j = 1; j <= order; ++j) {
            log << ",x" << j;
        }
        log << "]";
        for (size_t j = 0; j < order; ++j) {
            log << "*(x-x" << j << ")";
        }
    }
    log << "\n\n";

    log << "P(" << xStar << ") = " << dd[0][0];
    for (size_t order = 1; order < n; ++order) {
        double product = 1.0;
        for (size_t j = 0; j < order; ++j) {
            product *= (xStar - x[j]);
        }

        double term = dd[0][order] * product;
        result += term;

        log << " + (" << dd[0][order] << ")";
        for (size_t j = 0; j < order; ++j) {
            log << "*((" << xStar << " - " << x[j] << ")";
        }
        log << " = term " << term << "\n";
    }

    log << "\nP(" << xStar << ") = " << result << "\n\n";
    return result;
}
```

Результат:

=== Задача 3.1. Интерполяция ===

Узлы интерполяции:

```
i = 0, x_i = 0.3141592654, y_i = 0.3090169944
i = 1, x_i = 0.6283185307, y_i = 0.5877852523
i = 2, x_i = 0.9424777961, y_i = 0.8090169944
i = 3, x_i = 1.2566370614, y_i = 0.9510565163
```

Точка $x^* = 0.7853981634$

Точное значение $f(x^*) = \sin(x^*) = 0.7071067812$

=== Значение интерполяционного многочлена Лагранжа ===
0.7069466694

=== Значение интерполяционного многочлена Ньютона ===
0.7069466694

=== Фактическая абсолютная погрешность ===

$$|f(x^*) - L(x^*)| = 0.0001601118$$

$$|f(x^*) - P(x^*)| = 0.0001601118$$

=== Априорная оценка погрешности ===
 $\text{eps}(x^*) \leq M_{(n+1)} / (n+1)! * |\omega(x^*)|$
 Для $\sin(x)$: $\max |f^{(n+1)}(x)| \leq 1$
 Оценка = 0.0002283026

Сплайн

Использование одной интерполяционной формулы на большом числе узлов нецелесообразно. Интерполяционный многочлен может проявить свои колебательные свойства, его значения между узлами могут сильно отличаться от значений интерполируемой функции. Одна из возможностей преодоления этого недостатка заключается в применении сплайн-интерполяции. Суть сплайн-интерполяции заключается в определении интерполирующей функции по формулам одного типа для различных непересекающихся промежутков и в стыковке значений функции и её производных на их границах.

Наиболее широко применяемым является случай, когда между любыми двумя точками разбиения исходного отрезка строится многочлен n -й степени:

$$S(x) = \sum_{k=0}^n a_{ik} x^k, x_{i-1} \leq x \leq x_i, \quad i = \overline{1, n}$$

который в узлах интерполяции принимает значения аппроксимируемой функции и непрерывен вместе со своими $(n - 1)$ производными. Такой кусочно-непрерывный интерполяционный многочлен называется сплайном. Его коэффициенты находятся из условий равенства в узлах сетки значений сплайна и приближаемой функции, а также равенства $(n - 1)$ производных соответствующих многочленов. На практике наиболее часто используется интерполяционный многочлен третьей степени, который удобно представить как

$$S(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3,$$

$$x_{i-1} \leq x \leq x_i, \quad i = \overline{1, n}$$

Для построения кубического сплайна необходимо построить n многочленов третьей степени, т.е. определить $4n$ неизвестных a_i, b_i, c_i, d_i . Эти коэффициенты ищутся из условий в узлах сетки.

Входные данные: $X^* = 1.5$

i	0	1	2	3	4
x_i	0.0	1.0	2.0	3.0	4.0

f_i	0.0	0.5	0.86603	1.0	0.86603
-------	-----	-----	---------	-----	---------

Построение сплайна:

```

CubicSpline buildNaturalCubicSpline(const std::vector<double>& x,
                                     const std::vector<double>& y,
                                     std::ostream& log) {
    validateSplineInput(x, y);

    const size_t n = x.size() - 1; // число интервалов
    std::vector<double> h(n);

    log << "=== Построение кубического сплайна ===\n";
    log << "Граничные условия: S'(x0)=0, S'(xn)=0\n\n";

    for (size_t i = 0; i < n; ++i) {
        h[i] = x[i + 1] - x[i];
        log << "h[" << i << "] = x[" << (i + 1) << "] - x[" << i << "] = " << h[i] <<
"\n";
    }
    log << "\n";

    std::vector<double> cFull(n + 1, 0.0);

    // Если только 2 точки, сплайн вырождается в линейную функцию на одном отрезке
    if (n == 1) {
        CubicSpline spline;
        CubicSplineSegment seg;
        seg.a = y[0];
        seg.b = (y[1] - y[0]) / h[0];
        seg.c = 0.0;
        seg.d = 0.0;
        seg.xLeft = x[0];
        seg.xRight = x[1];
        spline.segments.push_back(seg);

        log << "Только один интервал: сплайн линейный.\n";
        return spline;
    }

    // Система для внутренних c[1..n-1]
    // a[i] - нижняя диагональ, b[i] - главная, c[i] - верхняя, d[i] - правая часть
    const size_t m = n - 1;
    std::vector<double> lower(m > 0 ? m - 1 : 0, 0.0);
    std::vector<double> diag(m, 0.0);
    std::vector<double> upper(m > 0 ? m - 1 : 0, 0.0);
    std::vector<double> rhs(m, 0.0);

    log << "Составляем трёхдиагональную систему для внутренних коэффициентов c_i:\n\n";

    for (size_t row = 0; row < m; ++row) {
        size_t i = row + 1; // внутренний индекс c_i, i=1..n-1

        if (row > 0) {
            lower[row - 1] = h[i - 1];
        }

        diag[row] = 2.0 * (h[i - 1] + h[i]);

        if (row + 1 < m) {
            upper[row] = h[i];
        }

        rhs[row] = 3.0 * ((y[i + 1] - y[i]) / h[i] - (y[i] - y[i - 1]) / h[i - 1]);

        log << "Уравнение для c[" << i << "]: ";
        if (i > 1) {

```

```

        log << h[i - 1] << " * c[" << (i - 1) << "]" + ";
    }
    log << 2.0 * (h[i - 1] + h[i]) << " * c[" << i << "]" + ";
    if (i < n - 1) {
        log << " + " << h[i] << " * c[" << (i + 1) << "]" + ";
    }
    log << " = " << rhs[row] << "\n";
}
log << "\n";

auto cInner = tridiagSolve(lower, diag, upper, rhs, log);

for (size_t i = 1; i <= n - 1; ++i) {
    cFull[i] = cInner[i - 1];
}
cFull[0] = 0.0;
cFull[n] = 0.0;

log << "\nНайденные коэффициенты c_i:\n";
for (size_t i = 0; i <= n; ++i) {
    log << "c[" << i << "] = " << cFull[i] << "\n";
}
log << "\n";

CubicSpline spline;
spline.segments.resize(n);

for (size_t i = 0; i < n; ++i) {
    CubicSplineSegment seg;
    seg.a = y[i];
    seg.b = (y[i + 1] - y[i]) / h[i] - h[i] * (2.0 * cFull[i] + cFull[i + 1]) / 3.0;
    seg.c = cFull[i];
    seg.d = (cFull[i + 1] - cFull[i]) / (3.0 * h[i]);
    seg.xLeft = x[i];
    seg.xRight = x[i + 1];

    spline.segments[i] = seg;

    log << "Интервал [" << seg.xLeft << ", " << seg.xRight << "]\n";
    log << "S_" << i << "(x) = a + b(x-x_i) + c(x-x_i)^2 + d(x-x_i)^3\n";
    log << "a = " << seg.a << "\n";
    log << "b = " << seg.b << "\n";
    log << "c = " << seg.c << "\n";
    log << "d = " << seg.d << "\n\n";
}

return spline;
}

```

Результат:

=== Задача 3.2. Кубический сплайн ===

Узлы интерполяции:

```

i = 0, x_i = 0.0000000000, f_i = 0.0000000000
i = 1, x_i = 1.0000000000, f_i = 0.5000000000
i = 2, x_i = 2.0000000000, f_i = 0.8660300000
i = 3, x_i = 3.0000000000, f_i = 1.0000000000
i = 4, x_i = 4.0000000000, f_i = 0.8660300000

```

=== Коэффициенты сплайна по интервалам ===

Интервал [0.0000000000, 1.0000000000]

```

a = 0.0000000000
b = 0.5240937500
c = 0.0000000000
d = -0.0240937500

```

Интервал [1.0000000000, 2.0000000000]

```

a = 0.5000000000
b = 0.4518125000
c = -0.0722812500
d = -0.0135012500

```

```

Интервал [2.0000000000, 3.0000000000]
a = 0.8660300000
b = 0.2667462500
c = -0.1127850000
d = -0.0199912500

```

```

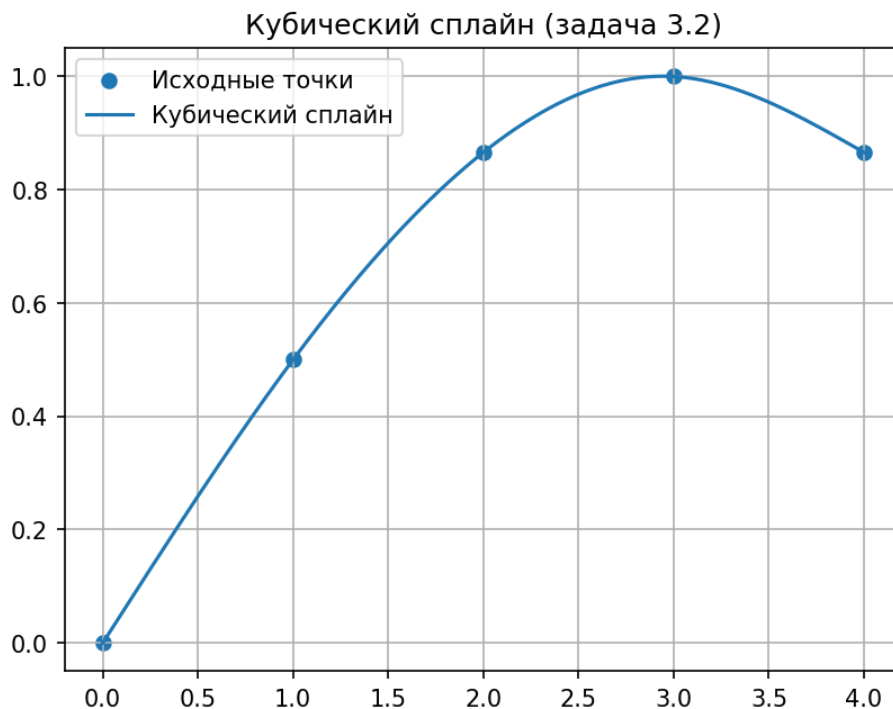
Интервал [3.0000000000, 4.0000000000]
a = 1.0000000000
b = -0.0187975000
c = -0.1727587500
d = 0.0575862500

```

```

=== Значение в точке x* ===
x* = 1.5000000000
S(x*) = 0.7061482812

```



Метод наименьших квадратов

Пусть задана таблично в узлах x_j функция $y_i = f(x_i)$. При этом значения функции y_i определены с некоторой погрешностью, также из физических соображений известен вид функции, которой должны приближенно удовлетворять табличные точки, например: многочлен степени n , у которого неизвестны коэффициенты a_i , $F_n(x) = \sum_{i=0}^n a_i x^i$. Неизвестные коэффициенты будем находить из условия минимума квадратичного отклонения многочлена от таблично заданной функции.

$$\Phi = \sum_{j=0}^n [F_n(x_j) - y_i]^2$$

Минимума Φ можно добиться только за счет изменения коэффициентов многочлена $F_n(x)$. Необходимые условия экстремума имеют вид

$$\frac{\partial \Phi}{\partial a_k} = 2 \sum_{j=0}^N \left[\sum_{i=0}^n a_i x_j^i - y_j \right] x_j^k = 0, \quad k = \overline{0, n}$$

Эту систему для удобства преобразуют к следующему виду:

$$\sum_{i=0}^n a_i \sum_{j=0}^N x_j^{k+1} = \sum_{j=0}^N y_j x_j^k, \quad k = \overline{0, n}$$

Эта система называется нормальной системой метода наименьших квадратов (МНК) и представляет собой систему линейных алгебраических уравнений относительно коэффициентов a_i . Решив систему, построим многочлен $F_n(x)$, приближающий функцию $f(x)$ и минимизирующий квадратичное отклонение.

Входные данные:

i	0	1	2	3	4	5
x_i	-1.0	0.0	1.0	2.0	3.0	4.0
y_i	-0.5	0.0	0.5	0.86603	1.0	0.86603

Метод наименьших квадратов:

```
PolynomialFit leastSquaresFit(const std::vector<double>& x,
                             const std::vector<double>& y,
                             int degree,
                             std::ostream& log) {
    validateInput(x, y);

    if (degree < 0) {
        throw std::invalid_argument("Степень полинома должна быть неотрицательной");
    }

    const int m = degree + 1;
    if (static_cast<size_t>(m) > x.size()) {
        throw std::invalid_argument("Слишком высокая степень полинома для данного числа точек");
    }

    log << "=== Метод наименьших квадратов, степень " << degree << " ===\n";

    std::vector<double> powerSums = computePowerSums(x, 2 * degree);
    std::vector<double> weightedSums = computeWeightedPowerSums(x, y, degree);

    log << "Суммы степеней x:\n";
    for (int k = 0; k <= 2 * degree; ++k) {
        log << "sum x^" << k << " = " << powerSums[k] << "\n";
    }
    log << "\n";

    log << "Суммы y*x^k:\n";
    for (int k = 0; k <= degree; ++k) {
```

```

        log << "sum y*x^" << k << " = " << weightedSums[k] << "\n";
    }
    log << "\n";

    Matrix A(m);
    std::vector<double> b(m, 0.0);

    for (int row = 0; row < m; ++row) {
        for (int col = 0; col < m; ++col) {
            A(row, col) = powerSums[row + col];
        }
        b[row] = weightedSums[row];
    }

    log << "Нормальная система:\n";
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < m; ++j) {
            log << std::setw(14) << A(i, j) << " ";
        }
        log << " | " << b[i] << "\n";
    }
    log << "\n";

    auto lu = luDecompose(A, log);
    std::vector<double> coeffs = solveLU(lu, b);

    log << "Коэффициенты полинома:\n";
    for (size_t i = 0; i < coeffs.size(); ++i) {
        log << "a[" << i << "] = " << coeffs[i] << "\n";
    }
    log << "Полином: " << formatPolynomial(coeffs) << "\n\n";

    PolynomialFit fit;
    fit.coeffs = coeffs;
    fit.valuesAtNodes.resize(x.size(), 0.0);
    fit.squaredError = 0.0;

    log << "Значения полинома в узлах и вклад в ошибку:\n";
    for (size_t i = 0; i < x.size(); ++i) {
        fit.valuesAtNodes[i] = evaluatePolynomial(coeffs, x[i]);
        double diff = y[i] - fit.valuesAtNodes[i];
        double sq = diff * diff;
        fit.squaredError += sq;

        log << "i = " << i
            << ", x = " << x[i]
            << ", y = " << y[i]
            << ", F(x) = " << fit.valuesAtNodes[i]
            << ", (y - F)^2 = " << sq << "\n";
    }

    log << "\nСумма квадратов ошибок Phi = " << fit.squaredError << "\n\n";
    return fit;
}

```

Результат:

=== Задача 3.3. Метод наименьших квадратов ===

Исходные данные:

```

i = 0, x_i = -1.0000000000, y_i = -0.5000000000
i = 1, x_i = 0.0000000000, y_i = 0.0000000000
i = 2, x_i = 1.0000000000, y_i = 0.5000000000
i = 3, x_i = 2.0000000000, y_i = 0.8660300000
i = 4, x_i = 3.0000000000, y_i = 1.0000000000
i = 5, x_i = 4.0000000000, y_i = 0.8660300000

```

=== Приближающий многочлен 1-й степени ===

$F1(x) = 0.0183641905 + 0.2913194286 \cdot x$
 Коэффициенты:
 $a[0] = 0.0183641905$
 $a[1] = 0.2913194286$
 Сумма квадратов ошибок $\Phi_{i1} = 0.2708179489$

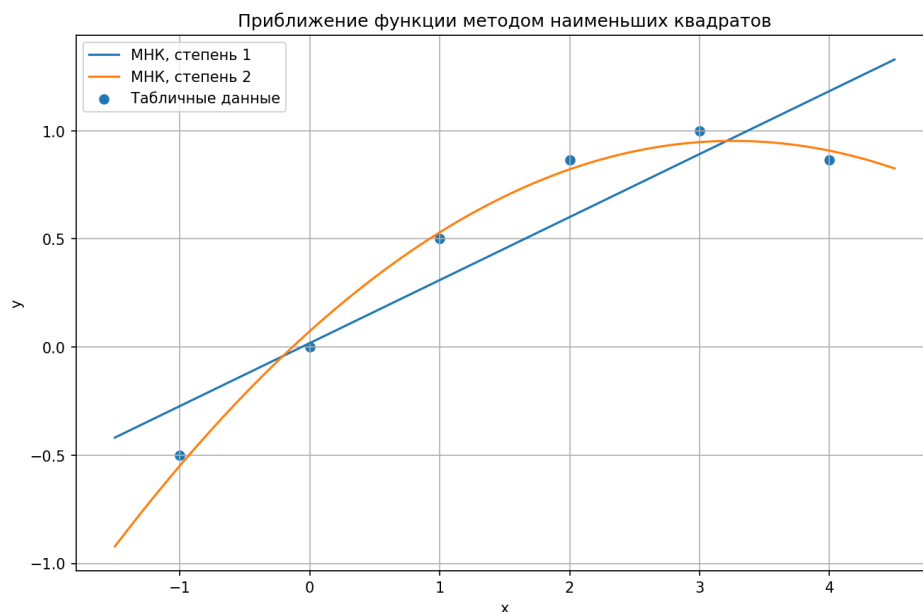
Значения $F1(x_i)$:
 $x = -1.0000000000$, $F1(x) = -0.2729552381$
 $x = 0.0000000000$, $F1(x) = 0.0183641905$
 $x = 1.0000000000$, $F1(x) = 0.3096836190$
 $x = 2.0000000000$, $F1(x) = 0.6010030476$
 $x = 3.0000000000$, $F1(x) = 0.8923224762$
 $x = 4.0000000000$, $F1(x) = 1.1836419048$

=== Приближающий многочлен 2-й степени ===
 $F2(x) = 0.0735305000 + 0.5395678214 \cdot x - 0.0827494643 \cdot x^2$
 Коэффициенты:
 $a[0] = 0.0735305000$
 $a[1] = 0.5395678214$
 $a[2] = -0.0827494643$
 Сумма квадратов ошибок $\Phi_{i2} = 0.0151789256$

Значения $F2(x_i)$:
 $x = -1.0000000000$, $F2(x) = -0.5487867857$
 $x = 0.0000000000$, $F2(x) = 0.0735305000$
 $x = 1.0000000000$, $F2(x) = 0.5303488571$
 $x = 2.0000000000$, $F2(x) = 0.8216682857$
 $x = 3.0000000000$, $F2(x) = 0.9474887857$
 $x = 4.0000000000$, $F2(x) = 0.9078103571$

=== Сравнение ошибок ===
 Полином 2-й степени даёт лучшее приближение по МНК.

=== PLOT_DATA ===
 N 6
 X -1.0000000000 0.0000000000 1.0000000000 2.0000000000 3.0000000000 4.0000000000
 Y -0.5000000000 0.0000000000 0.5000000000 0.8660300000 1.0000000000 0.8660300000
 C1 0.0183641905 0.2913194286
 C2 0.0735305000 0.5395678214 -0.0827494643



Численное дифференцирование

Формулы численного дифференцирования в основном используются при нахождении производных от функции $y = f(x)$, заданной таблично. Исходная функция $y_i = f(x_i)$ на отрезках $[x_j, x_{j+k}]$, заменяется некоторой приближающей, легко вычисляемой функцией $\varphi(x, \bar{a}), y = \varphi(x, \bar{a}) + R(x)$, где $R(x)$ – остаточный член приближения, $\bar{a}$ – набор коэффициентов, вообще говоря, различный для каждого из рассматриваемых отрезков, и полагают, что $y'(x) \approx \varphi'(x, \bar{a})$. Наиболее часто в качестве приближающей функции $\varphi(x, \bar{a})$ берется интерполяционный многочлен $\varphi(x, \bar{a}) = P_n(x)$, а производные соответствующих порядков определяются дифференцированием многочлена.

При решении практических задач, как правило, используются аппроксимации первых и вторых производных.

В первом приближении, таблично заданная функция может быть аппроксимирована отрезками прямой. В этом случае:

$$y' \approx \frac{y_{i+1} - y_i}{x_{i+1} - x_i} = \text{const}$$

При использовании для аппроксимации таблично заданной функции интерполяционного многочлена второй степени имеем:

$$y'(x) \approx \frac{y_{i+1} - y_i}{x_{i+1} - x_i} + \frac{\frac{y_{i+2} - y_{i+1}}{x_{i+2} - x_{i+1}} - \frac{y_{i+1} - y_i}{x_{i+1} - x_i}}{x_{i+2} - x_i} (2x - x_i - x_{i+1})$$

При равностоящих точках разбиения, данная формула обеспечивает второй порядок точности.

Для вычисления второй производной, необходимо использовать интерполяционный многочлен, как минимум второй степени. После дифференцирования многочлена получаем

$$y''(x) \approx 2 \frac{\frac{y_{i+2} - y_{i+1}}{x_{i+2} - x_{i+1}} - \frac{y_{i+1} - y_i}{x_{i+1} - x_i}}{x_{i+2} - x_i}$$

Входные данные: $X^* = 1.0$

i	0	1	2	3	4
x_i	-1.0	0.0	1.0	2.0	3.0
y_i	-0.5	0.0	0.50	0.86603	1.0

Дифференцирование в точке:

```

DerivativeResult differentiateAtPoint(const std::vector<double>& x,
                                     const std::vector<double>& y,
                                     double xStar,
                                     std::ostream& log) {

    validateInput(x, y);

    size_t i = findLeftIndex(x, xStar);

    const double x0 = x[i];
    const double x1 = x[i + 1];
    const double x2 = x[i + 2];

    const double y0 = y[i];
    const double y1 = y[i + 1];
    const double y2 = y[i + 2];

    const double h01 = x1 - x0;
    const double h12 = x2 - x1;
    const double h02 = x2 - x0;

    if (std::fabs(h01) < EPS || std::fabs(h12) < EPS || std::fabs(h02) < EPS) {
        throw std::runtime_error("Некорректные расстояния между узлами");
    }

    log << "=== Численное дифференцирование ===\n";
    log << "Используются точки:\n";
    log << "x0 = " << x0 << ", y0 = " << y0 << "\n";
    log << "x1 = " << x1 << ", y1 = " << y1 << "\n";
    log << "x2 = " << x2 << ", y2 = " << y2 << "\n";
    log << "x* = " << xStar << "\n\n";

    log << "Формула для первой производной:\n";
    log << "f'(x) ≈ (y1-y0)/(x1-x0)"
        << " + [ ((y2-y1)/(x2-x1) - (y1-y0)/(x1-x0)) / (x2-x0) ] * (2x - x0 - x1)\n\n";

    const double slope01 = (y1 - y0) / h01;
    const double slope12 = (y2 - y1) / h12;
    const double correction = (slope12 - slope01) / h02;

    const double firstDerivative =
        slope01 + correction * (2.0 * xStar - x0 - x1);

    log << "(y1 - y0)/(x1 - x0) = (" << y1 << " - " << y0 << ")/(" << x1 << " - " << x0
        << ") = " << slope01 << "\n";
    log << "(y2 - y1)/(x2 - x1) = (" << y2 << " - " << y1 << ")/(" << x2 << " - " << x1
        << ") = " << slope12 << "\n";
    log << "correction = (" << slope12 << " - " << slope01 << ")/(" << x2 << " - " << x0
        << ") = " << correction << "\n";
    log << "f'(x*) = " << slope01 << " + " << correction << " * (2*" << xStar
        << " - " << x0 << " - " << x1 << ") = " << firstDerivative << "\n\n";

    log << "Формула для второй производной:\n";
    log << "f''(x) ≈ 2 * [ ((y2-y1)/(x2-x1) - (y1-y0)/(x1-x0)) / (x2-x0) ]\n\n";

    const double secondDerivative = 2.0 * correction;

    log << "f''(x*) = 2 * " << correction << " = " << secondDerivative << "\n\n";

    DerivativeResult result;
    result.firstDerivative = firstDerivative;
    result.secondDerivative = secondDerivative;
    result.leftIndex = i;
    return result;
}

```

Результат:

=== Задача 3.4. Численное дифференцирование ===

Точка $x^* = 1.0000000000$
 Используются узлы:
 $x[1] = 0.0000000000$
 $x[2] = 1.0000000000$
 $x[3] = 2.0000000000$

=== Результат ===
 $f'(x^*) = 0.4330150000$
 $f''(x^*) = -0.1339700000$

Численное интегрирование

Формулы численного интегрирования используются в тех случаях, когда вычислить аналитически определенный интеграл $F = \int_a^b f(x)dx$ не удается. Рассмотрим наиболее простой и часто применяемый способ, когда подынтегральную функцию заменяют на интерполяционный многочлен.

При использовании интерполяционных многочленов различной степени, получают формулы численного интегрирования различного порядка точности.

Заменим подынтегральную функцию, интерполяционным многочленом Лагранжа нулевой степени, проходящим через середину отрезка – точку $\bar{x}_i = \frac{x_{i-1}+x_i}{2}$, получим **формулу прямоугольников**:

$$F = \int_a^b f(x)dx \approx \sum_{i=1}^N h_i f\left(\frac{x_{i-1} + x_i}{2}\right)$$

В случае таблично заданных функций удобно в качестве узлов интерполяции выбрать начало и конец отрезка интегрирования, т.е. заменить функцию $f(x)$ многочленом Лагранжа первой степени.

$$F = \int_a^b f(x)dx \approx \frac{1}{2} \sum_{i=1}^N (f_i + f_{i-1})h_i$$

Эта формула носит название **формулы трапеций**.

Для повышения порядка точности формулы численного интегрирования заменим подынтегральную кривую параболой – интерполяционным многочленом второй степени, выбрав в качестве узлов интерполяции концы и середину отрезка интегрирования: x_{i-1} , $x_{i-\frac{1}{2}} = \frac{x_{i-1}+x_i}{2}$, x_i .

Для случая $h_i = \frac{x_i - x_{i-1}}{2}$, получим **формулу Симпсона**:

$$F = \int_a^b f(x)dx \approx \frac{1}{3} \sum_{i=1}^N (f_{i-1} + 4f_{i-\frac{1}{2}} + f_i)h_i$$

Метод Рунге-Ромберга-Ричардсона позволяет получать более высокий порядок точности вычисления. Если имеются результаты вычисления определенного интеграла порядка точности p на сетке с шагом h : F_h и на сетке с шагом kh : F_{kh} , то

$$F = \int_a^b f(x)dx = F_h + \frac{F_h - F_{kh}}{k^p - 1} + O(h^{p+1})$$

Входные данные: $y = \frac{x}{2x+5}$, $X_0 = -1$, $X_k = 1$, $h_1 = 0.5$, $h_2 = 0.25$

Метод прямоугольника:

```
double integrateRectangles(const std::function<double(double)>& f,
                           double a,
                           double b,
                           double h,
                           std::ostream& log) {
    int n = computeStepCount(a, b, h);

    log << "=== Метод прямоугольников, h = " << h << " ===\n";

    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        double xl = a + i * h;
        double xr = xl + h;
        double xm = 0.5 * (xl + xr);
        double fxm = f(xm);
        double term = h * fxm;
        sum += term;

        log << "i = " << i
            << ", [" << xl << ", " << xr << "]"
            << ", mid = " << xm
            << ", f(mid) = " << fxm
            << ", term = " << term << "\n";
    }

    log << "Итог: I_rect = " << sum << "\n\n";
    return sum;
}
```

Метод трапеции:

```
double integrateTrapezoids(const std::function<double(double)>& f,
                           double a,
                           double b,
                           double h,
                           std::ostream& log) {
    int n = computeStepCount(a, b, h);

    log << "=== Метод трапеций, h = " << h << " ===\n";

    double sum = 0.0;
    for (int i = 0; i <= n; ++i) {
        double xi = a + i * h;
```

```

double fi = f(xi);
double coeff = (i == 0 || i == n) ? 1.0 : 2.0;
sum += coeff * fi;

log << "i = " << i
    << ", x = " << xi
    << ", f(x) = " << fi
    << ", coeff = " << coeff << "\n";
}

double result = 0.5 * h * sum;
log << "Итог: I_trap = h/2 * sum = " << result << "\n\n";
return result;
}

```

Метод Симпсона:

```

double integrateSimpson(const std::function<double(double)>& f,
                        double a,
                        double b,
                        double h,
                        std::ostream& log) {
    int n = computeStepCount(a, b, h);
    if (n % 2 != 0) {
        throw std::invalid_argument("Для метода Симпсона число интервалов должно быть чётным");
    }

    log << "=== Метод Симпсона, h = " << h << " ===\n";

    log << "i = 0, x = " << a << ", f(x) = " << f(a) << ", coeff = 1\n";

    double sum = 0.0;
    int pairCount = n / 2;

    for (int k = 0; k < pairCount; ++k) {
        double x0 = a + (2 * k) * h;
        double x1 = x0 + h;
        double x2 = x0 + 2 * h;

        double f0 = f(x0);
        double f1 = f(x1);
        double f2 = f(x2);

        double term = f0 + 4.0 * f1 + f2;
        sum += term;

        log << "k = " << k
            << ": [" << x0 << ", " << x2 << "], midpoint = " << x1 << "\n";
        log << "term = (" << h << "/3) * ("
            << f0 << " + 4*" << f1 << " + " << f2
            << ") = " << term << "\n";
    }

    double result = (h / 3.0) * sum;
    log << "Итог: I_simp = h/3 * sum = " << result << "\n\n";
    return result;
}

```

Метод Рунге-Ромберга-Ричардсона:

```

double rungeRomberg(double FhCoarse,
                    double FhFine,
                    double hCoarse,
                    double hFine,
                    int p) {
    if (hCoarse <= 0.0 || hFine <= 0.0) {

```

```

        throw std::invalid_argument("Шаги должны быть положительными");
    }
    if (p <= 0) {
        throw std::invalid_argument("Порядок точности p должен быть положительным");
    }

    double k = hCoarse / hFine;
    if (k <= 1.0 + EPS) {
        throw std::invalid_argument("Для Рунге-Ромберга требуется hCoarse > hFine");
    }

    return FhFine + (FhFine - FhCoarse) / (std::pow(k, p) - 1.0);
}

```

Результат:

=== Задача 3.5. Численное интегрирование ===

$f(x) = x / (2x + 5)$
Интервал: $[-1.0000000000, 1.0000000000]$
 $h1 = 0.5000000000$
 $h2 = 0.2500000000$

=== Результаты для $h1$ ===

Метод прямоугольников: -0.0545010545
Метод трапеций: -0.0684523810
Метод Симпсона: -0.0595238095

=== Результаты для $h2$ ===

Метод прямоугольников: -0.0579479937
Метод трапеций: -0.0614767177
Метод Симпсона: -0.0591514967

=== Теоретические оценки погрешностей ===

Метод прямоугольников: для $h1$ 0.0154320988 , для $h2$ 0.0038580247
Метод трапеций: для $h1$ 0.0154320988 , для $h2$ 0.0038580247
Метод Симпсона: для $h1$ 0.0154320988 , для $h2$ 0.0038580247

=== Уточнение по Рунге-Ромбергу ===

Прямоугольники ($p=2$): -0.0590969734
Трапеции ($p=2$): -0.0591514967
Симпсон ($p=4$): -0.0591266758

=== Точное значение интеграла ===

-0.0591223255

=== Абсолютная погрешность уточнённых значений ===

Прямоугольники: 0.0000253521
Трапеции: 0.0000291712
Симпсон: 0.0000043503

Лабораторная работа №4

О программе

Программа написана на языке C++ с использованием стандарта C++23. Решение реализовано модульно: задачи Коши и краевые задачи для обыкновенных дифференциальных уравнений вынесены в отдельные классы и исходные файлы. Для решения вспомогательных систем используется метод прогонки, реализованный в первой лабораторной работе.

Инструкция к запуску

Проект собирается с помощью CMake. Все входные данные хранятся в директории input в формате txt. При запуске программы последовательно выполняются все задания лабораторной работы, результаты сохраняются в директорию output.

Пример для выполнения всех заданий:

```
cmake --build build
./main
```

Численные методы решения задачи Коши

Рассматривается задача Коши для одного дифференциального уравнения первого порядка разрешенного относительно производной

$$y' = f(x, y)$$

$$y(x_0) = y_0$$

Требуется найти решение на отрезке $[a, b]$, где $x_0 = a$.

Введем разностную сетку на отрезке $[a, b]$: $\Omega^{(k)} = \{x_k = x_0 + hk\}$.

Точки x_i называются узлами разностной сетки, расстояния между узлами — *шагом разностной сетки*, а совокупность значений какой-либо величины, заданных в узлах сетки называется *сеточной функцией*.

Приближенное решение задачи Коши будем искать численно в виде сеточной функции.

Метод Эйлера (явный):

Метод Эйлера играет важную роль в теории численных методов решения ОДУ, хотя и не часто используется в практических расчетах из-за невысокой точности. Вывод расчетных соотношений для этого метода может быть произведен несколькими способами: с помощью геометрической

интерпретации, с использованием разложения в ряд Тейлора, конечно, разностным методом (с помощью разностной аппроксимации производной), квадратурным способом (использованием эквивалентного интегрального уравнения).

Рассмотрим вывод соотношений метода Эйлера геометрическим способом. Решение в узле x_0 известно из начальных условий. Рассмотрим процедуру получения решения в узле x_1 .

График функции $y^{(h)}$, которая является решением задачи Коши, представляет собой гладкую кривую, проходящую через точку (x_0, y_0) , согласно условию $y(x_0) = y_0$, и имеет в этой точке касательную. Тангенс угла наклона касательной к оси Ox равен значению производной от решения в точке x_0 и равен значению правой части дифференциального уравнения в точке (x_0, y_0) согласно выражению $y'(x_0) = f(x_0, y_0)$. В случае небольшого шага разностной сетки h график функции и график касательной не успевают сильно разойтись друг от друга и можно в качестве значения решения в узле x_1 принять значение касательной y_1 , вместо значения неизвестного точного решения $y_{1\text{ист}}$. При этом допускается погрешность $|y_1 - y_{1\text{ист}}|$ геометрически представленная отрезком CD . Из прямоугольного треугольника ABC находим $CB = BA \cdot \operatorname{tg}(CAB)$ или $\Delta y = h y'(x_0)$. Учитывая, что $\Delta y = y_1 - y_0$ и заменяя производную $y'(x_0)$ на правую часть дифференциального уравнения, получаем соотношение $y_1 = y_0 + h f(x_0, y_0)$. Считая теперь точку (x_1, y_1) начальной и повторяя все предыдущие рассуждения, получим значение y_2 в узле x_2 .

Переход к произвольным индексам дает формулу метода Эйлера:

$$y_{k+1} = y_k + h f(x_k, y_k)$$

Неявный метод Эйлера:

Неявный метод Эйлера (также известный как метод трапеций) является численным методом решения задачи Коши для обыкновенного дифференциального уравнения первого порядка. Он представляет собой один из классических методов решения дифференциальных уравнений, который имеет высокую точность и устойчивость при большом шаге интегрирования.

Идея метода заключается в замене производной функции на приближенную, используя формулу для среднего значения. В результате получается неявное уравнение, которое решается численно.

Метод Рунге-Кутты:

Семейство явных методов Рунге-Кутты p -го порядка записывается в виде совокупности формул:

$$y_{k+1} = y_k + \Delta y_k$$

$$\Delta y_k = \sum_{i=1}^p c_i K_i^k$$

$$K_i^k = hf(x_k + a_i h, y_k + h \sum_{j=1}^{i-1} b_{ij} K_j^k)$$

$$i = \overline{2, p}$$

Метод Рунге-Кутты четвертого порядка:

$$a = \left(0, \frac{1}{2}, \frac{1}{2}, 1\right); b = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \end{pmatrix}; c = \left(\frac{1}{6}, \frac{1}{3}, \frac{1}{3}, \frac{1}{6}\right)$$

$$\Delta y_k = \frac{1}{6} (K_1^k + K_2^k + K_3^k + K_4^k)$$

$$K_1^k = hf(x_k, y_k); K_2^k = hf(x_k + 0.5h, y_k + 0.5K_1^k)$$

$$K_3^k = hf(x_k + 0.5h, y_k + 0.5K_2^k); K_4^k = hf(x_k + h, y_k + K_3^k)$$

Метод Адамса:

При использовании интерполяционного многочлена 3-ей степени построенного по значениям подынтегральной функции в последних четырех узлах получим метод Адамса четвертого порядка точности:

$$y_{k+1} = y_k + \frac{h}{24} (55f_k - 59f_{k-1} + 37f_{k-2} - 9f_{k-3})$$

Метод Адамса как и все многошаговые методы не является самостартующим, то есть для того, что бы использовать метод Адамса необходимо иметь решения в первых четырех узлах. В узле x_0 решение y_0 известно из начальных условий, а в других трех узлах x_1, x_2, x_3 решения y_1, y_2, y_3 можно получить с помощью подходящего одношагового метода, например: метода Рунге-Кутты четвертого порядка.

Решение задачи Коши для системы обыкновенных дифференциальных уравнений:

Рассматривается задача Коши для системы дифференциальных уравнений первого порядка разрешенных относительно производной

$$\begin{cases} y_1' = f_1(x, y, y_1, y_2, \dots, y_n) \\ y_2' = f_2(x, y, y_1, y_2, \dots, y_n) \\ \dots \\ y_n' = f_n(x, y, y_1, y_2, \dots, y_n) \end{cases}$$

$$y_1(x_0) = y_{01}$$

$$y_2(x_0) = y_{02}$$

...

$$y_n(x_0) = y_{0n}$$

К системе дифференциальных уравнений можно применить все методы рассмотренные выше. Уравнения решаются по порядку.

Для задачи Коши 2-го порядка $y'' = f(x, y, y')$ можно применить следующее разложение в систему (используя замену $z = y'(x)$):

$$\begin{cases} z' = f(x, y, z) \\ y' = z(x) \end{cases}$$

$$y(x_0) = y_0$$

$$y'(x_0) = z_0$$

Входные данные:

$y'' + y - \sin 3x = 0,$ $y(0) = 1,$ $y'(0) = 1,$ $x \in [0, 1], h = 0.1$	$y = \cos x + \frac{11}{8} \sin x - \frac{\sin 3x}{8}$
--	--

Метод Эйлера:

```
ODESolution solveEuler(double x0, double xk, double y0, double z0, double h, std::ostream& log) {
    int n = computeStepCount(x0, xk, h);

    ODESolution solution;
    solution.x.resize(n + 1);
    solution.y.resize(n + 1);
    solution.z.resize(n + 1);

    solution.x[0] = x0;
    solution.y[0] = y0;
    solution.z[0] = z0;

    log << "=== Метод Эйлера (явный), h = " << h << " ===\n";
    log << "k = 0, x = " << solution.x[0]
        << ", y = " << solution.y[0]
        << ", z = " << solution.z[0] << "\n";

    for (int i = 0; i < n; ++i) {
```

```

double x = solution.x[i];
double y = solution.y[i];
double z = solution.z[i];

solution.x[i + 1] = x + h;
solution.y[i + 1] = y + h * z;
solution.z[i + 1] = z + h * rhs(x, y);

log << "k = " << (i + 1)
    << ", x = " << solution.x[i + 1]
    << ", y = " << solution.y[i + 1]
    << ", z = " << solution.z[i + 1]
    << ", |dy| = " << std::fabs(solution.y[i + 1] - y)
    << "\n";
}

log << "\n";
return solution;
}

```

Метод Рунге-Кутты:

```

ODESolution solveRungeKutta4(double x0, double xk, double y0, double z0, double h,
std::ostream& log) {
    int n = computeStepCount(x0, xk, h);

    ODESolution solution;
    solution.x.resize(n + 1);
    solution.y.resize(n + 1);
    solution.z.resize(n + 1);

    solution.x[0] = x0;
    solution.y[0] = y0;
    solution.z[0] = z0;

    log << "=== Метод Рунге-Кутты 4-го порядка, h = " << h << " ===\n";
    log << "k = 0, x = " << solution.x[0]
        << ", y = " << solution.y[0]
        << ", z = " << solution.z[0] << "\n";

    for (int i = 0; i < n; ++i) {
        double x = solution.x[i];
        double y = solution.y[i];
        double z = solution.z[i];

        double K1 = h * z;
        double L1 = h * rhs(x, y);

        double K2 = h * (z + L1 / 2.0);
        double L2 = h * rhs(x + h / 2.0, y + K1 / 2.0);

        double K3 = h * (z + L2 / 2.0);
        double L3 = h * rhs(x + h / 2.0, y + K2 / 2.0);

        double K4 = h * (z + L3);
        double L4 = h * rhs(x + h, y + K3);

        solution.x[i + 1] = x + h;
        solution.y[i + 1] = y + (K1 + 2.0 * K2 + 2.0 * K3 + K4) / 6.0;
        solution.z[i + 1] = z + (L1 + 2.0 * L2 + 2.0 * L3 + L4) / 6.0;

        log << "k = " << (i + 1)
            << ", x = " << solution.x[i + 1]
            << ", y = " << solution.y[i + 1]
            << ", z = " << solution.z[i + 1]
            << ", K = (" << K1 << ", " << K2 << ", " << K3 << ", " << K4 << ")"
            << ", L = (" << L1 << ", " << L2 << ", " << L3 << ", " << L4 << ")"
            << "\n";
    }
}

```

```

    }

    log << "\n";
    return solution;
}

```

Метод Адамса:

```

ODESolution solveAdams4(double x0, double xk, double y0, double z0, double h, std::os-
tream& log) {
    int n = computeStepCount(x0, xk, h);
    if (n < 4) {
        throw std::invalid_argument("Для метода Адамса 4-го порядка требуется не менее 4
шагов");
    }

    log << "=== Метод Адамса 4-го порядка, h = " << h << " ===\n";
    log << "Первые четыре точки получаем методом Рунге-Кутты 4-го порядка\n";

    ODESolution solution = solveRungeKutta4(x0, x0 + 3.0 * h, y0, z0, h, log);
    solution.x.resize(n + 1);
    solution.y.resize(n + 1);
    solution.z.resize(n + 1);
    for (int i = 4; i <= n; ++i) {
        solution.x[i] = x0 + i * h;
    }

    for (int i = 3; i < n; ++i) {
        double f0 = rhs(solution.x[i], solution.y[i]);
        double f1 = rhs(solution.x[i - 1], solution.y[i - 1]);
        double f2 = rhs(solution.x[i - 2], solution.y[i - 2]);
        double f3 = rhs(solution.x[i - 3], solution.y[i - 3]);

        solution.y[i + 1] = solution.y[i] + h * (
            55.0 * solution.z[i]
            - 59.0 * solution.z[i - 1]
            + 37.0 * solution.z[i - 2]
            - 9.0 * solution.z[i - 3]
        ) / 24.0;

        solution.z[i + 1] = solution.z[i] + h * (
            55.0 * f0
            - 59.0 * f1
            + 37.0 * f2
            - 9.0 * f3
        ) / 24.0;

        log << "k = " << (i + 1)
            << ", x = " << solution.x[i + 1]
            << ", y = " << solution.y[i + 1]
            << ", z = " << solution.z[i + 1]
            << ", f = (" << f3 << ", " << f2 << ", " << f1 << ", " << f0 << ")"
            << "\n";
    }

    log << "\n";
    return solution;
}

```

Результат:

=== Задача 4.1. Решение задачи Коши ===

Точное решение: $y = \cos(x) + 11/8 \sin(x) - 1/8 \sin(3x)$
 Интервал: [0.0000000000, 1.0000000000]
 $h_1 = 0.1000000000$, $h_2 = 0.0500000000$

В обозначениях метода Рунге-Кутты: K_i -- приращения для y , L_i -- приращения для $z = y'$.

=== Метод Эйлера (h1) ===

x	y z - z_exact	y_exact	y - y_exact		z = y'	z_exact
0.0000000000	1.0000000000 0.0000000000	1.0000000000	0.0000000000	1.0000000000	1.0000000000	1.0000000000
0.1000000000	1.1000000000 0.0100461272	1.0953350873	0.0046649127	0.9000000000	0.9100461272	0.9100461272
0.2000000000	1.1900000000 0.0198693375	1.1826565985	0.0073434015	0.8195520207	0.8394213581	0.8394213581
0.3000000000	1.2719552021 0.0279474598	1.2637609096	0.0081942925	0.7570162680	0.7849637278	0.7849637278
0.4000000000	1.3476568289 0.0330029278	1.3400063289	0.0076504999	0.7081534388	0.7411563665	0.7411563665
0.5000000000	1.4184721727 0.0341323689	1.4121058041	0.0063663686	0.6665916645	0.7007240334	0.7007240334
0.6000000000	1.4851313392 0.0309008368	1.4799880620	0.0051432772	0.6244939459	0.6553947826	0.6553947826
0.7000000000	1.5475807338 0.0233920345	1.5427403364	0.0048403974	0.5733655750	0.5967576095	0.5967576095
0.8000000000	1.6049172913 0.0122098395	1.5986384368	0.0062788545	0.5049284383	0.5171382778	0.5171382778
0.9000000000	1.6554101351 0.0015691772	1.6452619840	0.0101481511	0.4119830272	0.4104138500	0.4104138500
1.0000000000	1.6966084378 0.0164881298	1.6796849090	0.0169235289	0.2891800018	0.2726918720	0.2726918720

=== Метод Эйлера (h2) ===

x	y z - z_exact	y_exact	y - y_exact		z = y'	z_exact
0.0000000000	1.0000000000 0.0000000000	1.0000000000	0.0000000000	1.0000000000	1.0000000000	1.0000000000
0.0500000000	1.0500000000 0.0025132845	1.0487918516	0.0012081484	0.9500000000	0.9525132845	0.9525132845
0.1000000000	1.0975000000 0.0050742206	1.0953350873	0.0021649127	0.9049719066	0.9100461272	0.9100461272
0.1500000000	1.1427485953 0.0075815193	1.1398778183	0.0028707770	0.8648729170	0.8724544363	0.8724544363
0.2000000000	1.1859922412 0.0099375942	1.1826565985	0.0033356427	0.8294837639	0.8394213581	0.8394213581
0.2500000000	1.2274664294 0.0120510193	1.2238880207	0.0035784087	0.7984162755	0.8104672948	0.8104672948
0.3000000000	1.2673872431 0.0138388357	1.2637609096	0.0036263336	0.7711248920	0.7849637278	0.7849637278
0.3500000000	1.3059434878 0.0152286544	1.3024292949	0.0035141929	0.7469218754	0.7621505298	0.7621505298
0.4000000000	1.3432895815 0.0161605043	1.3400063289	0.0032832526	0.7249958623	0.7411563665	0.7411563665
0.4500000000	1.3795393746 0.0165883865	1.3765592920	0.0029800826	0.7044333375	0.7210217240	0.7210217240
0.5000000000	1.4147610415 0.0164814967	1.4121058041	0.0026552374	0.6842425366	0.7007240334	0.7007240334
0.5500000000	1.4489731683 0.0158250883	1.4466113333	0.0023618351	0.6633792339	0.6792043222	0.6792043222
0.6000000000	1.4821421300 0.0146209557	1.4799880620	0.0021540681	0.6407738269	0.6553947826	0.6553947826
0.6500000000	1.5141808214 0.0128875271	1.5120951421	0.0020856793	0.6153591019	0.6282466290	0.6282466290
0.7000000000	1.5449487765 0.0106595629	1.5427403364	0.0022084401	0.5860980466	0.5967576095	0.5967576095
0.7500000000	1.5742536788 0.0079874671	1.5716830143	0.0025706645	0.5520110761	0.5599985432	0.5599985432
0.8000000000	1.6018542326 0.0049362257	1.5986384368	0.0032157958	0.5122020520	0.5171382778	0.5171382778
0.8500000000	1.6274643352 0.0015839967	1.6232832383	0.0041810969	0.4658824994	0.4674664962	0.4674664962

0.9000000000	1.6507584602	1.6452619840	0.0054964762	0.4123934685	0.4104138500
0.0019796185					
0.9500000000	1.6713781336	1.6641946570	0.0071834766	0.3512245395	0.3455689573
0.0056555822					
1.0000000000	1.6889393606	1.6796849090	0.0092544516	0.2820295335	0.2726918720
0.0093376615					

=== Метод Рунге-Кутты 4-го порядка (h1) ===

x	y	y_exact	y - y_exact	z = y'	z_exact
	z - z_exact				
0.0000000000	1.0000000000	1.0000000000	0.0000000000	1.0000000000	1.0000000000
0.0000000000					
0.1000000000	1.0953356271	1.0953350873	0.0000005398	0.9100462591	0.9100461272
0.0000001319					
0.2000000000	1.1826576519	1.1826565985	0.0000010534	0.8394216860	0.8394213581
0.0000003279					
0.3000000000	1.2637624036	1.2637609096	0.0000014941	0.7849643032	0.7849637278
0.0000005754					
0.4000000000	1.3400081520	1.3400063289	0.0000018230	0.7411572249	0.7411563665
0.0000008584					
0.5000000000	1.4121078170	1.4121058041	0.0000020129	0.7007251913	0.7007240334
0.0000011579					
0.6000000000	1.4799901122	1.4799880620	0.0000020502	0.6553962368	0.6553947826
0.0000014542					
0.7000000000	1.5427422733	1.5427403364	0.0000019369	0.5967593372	0.5967576095
0.0000017277					
0.8000000000	1.5986401267	1.5986384368	0.0000016900	0.5171402388	0.5171382778
0.0000019610					
0.9000000000	1.6452633242	1.6452619840	0.0000013402	0.4104159896	0.4104138500
0.0000021396					
1.0000000000	1.6796858382	1.6796849090	0.0000009293	0.2726941253	0.2726918720
0.0000022534					

=== Метод Рунге-Кутты 4-го порядка (h2) ===

x	y	y_exact	y - y_exact	z = y'	z_exact
	z - z_exact				
0.0000000000	1.0000000000	1.0000000000	0.0000000000	1.0000000000	1.0000000000
0.0000000000					
0.0500000000	1.0487918685	1.0487918516	0.0000000169	0.9525132879	0.9525132845
0.0000000034					
0.1000000000	1.0953351211	1.0953350873	0.0000000338	0.9100461350	0.9100461272
0.0000000078					
0.1500000000	1.1398778685	1.1398778183	0.0000000501	0.8724544495	0.8724544363
0.0000000132					
0.2000000000	1.1826566642	1.1826565985	0.0000000657	0.8394213777	0.8394213581
0.0000000196					
0.2500000000	1.2238881007	1.2238880207	0.0000000800	0.8104673215	0.8104672948
0.0000000268					
0.3000000000	1.2637610024	1.2637609096	0.0000000928	0.7849637625	0.7849637278
0.0000000347					
0.3500000000	1.3024293988	1.3024292949	0.0000001039	0.7621505729	0.7621505298
0.0000000432					
0.4000000000	1.3400064418	1.3400063289	0.0000001129	0.7411564186	0.7411563665
0.0000000521					
0.4500000000	1.3765594117	1.3765592920	0.0000001197	0.7210217854	0.7210217240
0.0000000614					
0.5000000000	1.4121059284	1.4121058041	0.0000001242	0.7007241042	0.7007240334
0.0000000708					
0.5500000000	1.4466114596	1.4466113333	0.0000001263	0.6792044025	0.6792043222
0.0000000803					
0.6000000000	1.4799881880	1.4799880620	0.0000001260	0.6553948722	0.6553947826
0.0000000895					
0.6500000000	1.5120952654	1.5120951421	0.0000001234	0.6282467276	0.6282466290
0.0000000985					
0.7000000000	1.5427404549	1.5427403364	0.0000001185	0.5967577166	0.5967576095
0.0000001071					
0.7500000000	1.5716831259	1.5716830143	0.0000001116	0.5599986583	0.5599985432
0.0000001151					

0.8000000000	1.5986385396	1.5986384368	0.0000001028	0.5171384001	0.5171382778
0.0000001223					
0.8500000000	1.6232833308	1.6232832383	0.0000000925	0.4674666250	0.4674664962
0.0000001288					
0.9000000000	1.6452620650	1.6452619840	0.0000000810	0.4104139843	0.4104138500
0.0000001343					
0.9500000000	1.6641947256	1.6641946570	0.0000000686	0.3455690962	0.3455689573
0.0000001389					
1.0000000000	1.6796849646	1.6796849090	0.0000000556	0.2726920143	0.2726918720
0.0000001424					

=== Метод Адамса 4-го порядка (h1) ===

x	y	y_exact	y - y_exact	z = y'	z_exact
	z - z_exact				
0.0000000000	1.0000000000	1.0000000000	0.0000000000	1.0000000000	1.0000000000
0.0000000000					
0.1000000000	1.0953356271	1.0953350873	0.0000005398	0.9100462591	0.9100461272
0.0000001319					
0.2000000000	1.1826576519	1.1826565985	0.0000010534	0.8394216860	0.8394213581
0.0000003279					
0.3000000000	1.2637624036	1.2637609096	0.0000014941	0.7849643032	0.7849637278
0.0000005754					
0.4000000000	1.3400910236	1.3400063289	0.0000846947	0.7409883566	0.7411563665
0.0001680100					
0.5000000000	1.4122145726	1.4121058041	0.0001087685	0.7002992514	0.7007240334
0.0004247820					
0.6000000000	1.4800771970	1.4799880620	0.0000891350	0.6546805448	0.6553947826
0.0007142378					
0.7000000000	1.5427507994	1.5427403364	0.0000104630	0.5957298248	0.5967576095
0.0010277847					
0.8000000000	1.5985056549	1.5986384368	0.0001327819	0.5158156929	0.5171382778
0.0013225849					
0.9000000000	1.6449318647	1.6452619840	0.0003301193	0.4088473211	0.4104138500
0.0015665289					
1.0000000000	1.6791125121	1.6796849090	0.0005723968	0.2709605708	0.2726918720
0.0017313012					

=== Метод Адамса 4-го порядка (h2) ===

x	y	y_exact	y - y_exact	z = y'	z_exact
	z - z_exact				
0.0000000000	1.0000000000	1.0000000000	0.0000000000	1.0000000000	1.0000000000
0.0000000000					
0.0500000000	1.0487918685	1.0487918516	0.0000000169	0.9525132879	0.9525132845
0.0000000034					
0.1000000000	1.0953351211	1.0953350873	0.0000000338	0.9100461350	0.9100461272
0.0000000078					
0.1500000000	1.1398778685	1.1398778183	0.0000000501	0.8724544495	0.8724544363
0.0000000132					
0.2000000000	1.1826596693	1.1826565985	0.0000030708	0.8394186440	0.8394213581
0.0000027142					
0.2500000000	1.2238936283	1.2238880207	0.0000056076	0.8104601309	0.8104672948
0.0000071638					
0.3000000000	1.2637686421	1.2637609096	0.0000077325	0.7849509115	0.7849637278
0.0000128163					
0.3500000000	1.3024385425	1.3024292949	0.0000092476	0.7621307379	0.7621505298
0.0000197918					
0.4000000000	1.3400163422	1.3400063289	0.0000100133	0.7411285388	0.7411563665
0.0000278277					
0.4500000000	1.3765692560	1.3765592920	0.0000099640	0.7209850062	0.7210217240
0.0000367178					
0.5000000000	1.4121148199	1.4121058041	0.0000090157	0.7006778032	0.7007240334
0.0000462302					
0.5500000000	1.4466184513	1.4466113333	0.0000071181	0.6791482127	0.6792043222
0.0000561096					
0.6000000000	1.4799923049	1.4799880620	0.0000042429	0.6553286897	0.6553947826
0.0000660929					
0.6500000000	1.5120955279	1.5120951421	0.0000003858	0.6281707191	0.6282466290
0.0000759099					

```

0.7000000000 1.5427359039 1.5427403364 0.0000044325 0.5966723189 0.5967576095
0.0000852906
0.7500000000 1.5716728471 1.5716830143 0.0000101672 0.5599045716 0.5599985432
0.0000939716
0.8000000000 1.5986216867 1.5986384368 0.0000167501 0.5170365756 0.5171382778
0.0001017022
0.8500000000 1.6232591477 1.6232832383 0.0000240906 0.4673582464 0.4674664962
0.0001082498
0.9000000000 1.6452299057 1.6452619840 0.0000320782 0.4103004442 0.4104138500
0.0001134058
0.9500000000 1.6641540725 1.6641946570 0.0000405845 0.3454519674 0.3455689573
0.0001169899
1.0000000000 1.6796354432 1.6796849090 0.0000494658 0.2725730170 0.2726918720
0.0001188550

```

=== Оценка погрешности ===

Метод Эйлера

```

max |y_h - y_exact|    = 0.0169235289
max |y_h/2 - y_exact|  = 0.0092544516
Runge-Romberg estimate = 0.0076690772

```

Метод Рунге-Кутты 4-го порядка

```

max |y_h - y_exact|    = 0.0000020502
max |y_h/2 - y_exact|  = 0.0000001263
Runge-Romberg estimate = 0.0000001283

```

Метод Адамса 4-го порядка

```

max |y_h - y_exact|    = 0.0005723968
max |y_h/2 - y_exact|  = 0.0000494658
Runge-Romberg estimate = 0.0000348621

```

Численные методы решения краевой задачи для ОДУ

Примером краевой задачи является двухточечная краевая задача для обыкновенного дифференциального уравнения второго порядка.

$$y'' = f(x, y, y')$$

с граничными условиями, заданными на концах отрезка $[a, b]$.

$$\begin{aligned} y(a) &= y_0 \\ y(b) &= y_1 \end{aligned} \quad \text{— граничные условия 1 рода}$$

Следует найти такое решение $y(x)$ на этом отрезке, которое принимает на концах отрезка значения y_0, y_1 .

Кроме граничных условий первого рода, используются еще условия на производные от решения на концах - граничные условия второго рода:

$$\begin{aligned} y'(a) &= \hat{y}_0 \\ y'(b) &= \hat{y}_1 \end{aligned}$$

или линейная комбинация решений и производных — граничные условия третьего рода:

$$\begin{aligned}\alpha y(a) + \beta y'(a) &= \hat{y}_0 \\ \delta y(b) + \gamma y'(b) &= \hat{y}_1\end{aligned}$$

Возможно на разных концах отрезка использовать условия различных типов.

Метод стрельбы:

Суть метода заключена в многократном решении задачи Коши для приближенного нахождения решения краевой задачи.

Пусть надо решить краевую задачу краевыми условиями 1-го рода на отрезке $[a, b]$. Вместо исходной задачи формулируется задача Коши с уравнением $y'' = f(x, y, y')$ и с начальными условиями

$$\begin{aligned}y(a) &= y_0 \\ y'(b) &= \eta\end{aligned}$$

Положим сначала некоторое начальное значение параметру $\eta = \eta_0$, после чего решим каким-либо методом задачу Коши. Пусть $y = y_0(x, y_0, \eta_0)$ решение этой задачи на интервале $[a, b]$, тогда сравнивая значение функции $y_0(b, y_0, \eta_0)$ со значением y_1 в правом конце отрезка можно получить информацию для корректировки угла наклона касательной к решению в левом конце отрезка. Задачу можно сформулировать таким образом: требуется найти такое значение переменной η^* , чтобы решение $y(b, y_0, \eta^*)$ в правом конце отрезка совпало со значением y_1 . Другими словами, решение исходной задачи эквивалентно нахождению корня уравнения

$$\Phi(\eta) = y(b, y_0, \eta) - y_1 = 0$$

Следующее значение искомого корня определяется по соотношению

$$\eta_{j+2} = \eta_{j+1} - \frac{\eta_{j+1} - \eta_j}{\Phi(\eta_{j+1}) - \Phi(\eta_j)} \Phi(\eta_{j+1})$$

Конечно-разностный метод:

Рассмотрим двухточечную краевую задачу для линейного дифференциального уравнения второго порядка на отрезке $[a, b]$

$$\begin{aligned}y'' + p(x)y' + q(x)y &= f(x) \\ y(a) &= y_0, y(b) = y_1\end{aligned}$$

Введем разностную аппроксимацию производных следующим образом:

$$y'_k = \frac{y_{k+1} - y_{k-1}}{2h} + O(h^2)$$

$$y_k'' = \frac{y_{k+1} - 2y_k + y_{k-1}}{h^2} + O(h^2)$$

Подставляя аппроксимации производных в уравнение, получим систему уравнений для нахождения y_k :

$$\begin{cases} y_0 = y_a \\ \frac{y_{k+1} - 2y_k + y_{k-1}}{h^2} + p(x_k) \frac{y_{k+1} - y_{k-1}}{2h} + q(x_k)y_k = f(x_k), \quad k = \overline{1, N-1} \\ y_N = y_b \end{cases}$$

Входные данные:

$xy'' + 2y' - xy = 0,$ $y'(1) = 0,$ $1.5y(2) + y'(2) = e^2$	$y(x) = \frac{e^x}{x}$
---	------------------------

Метод стрельбы:

```
BvpSolution solveShooting(const BvpInput& input, double h, std::ostream& log) {
    if (std::fabs(input.beta) < 1e-12) {
        throw std::invalid_argument("Для метода стрельбы по методичке требуется beta != 0");
    }

    log << "=== Метод стрельбы, h = " << h << " ===\n";
    log << "Решаем задачу Коши с параметром eta = y(a).\n";
    log << "Начальная производная выражается из левого граничного условия:\n";
    log << "y'(a) = (A - alpha * eta) / beta.\n";
    log << "Строим Phi(eta) = delta * y(b, eta) + gamma * y'(b, eta) - B\n";
    log << "и уточняем eta методом секущих.\n";

    double eta0 = input.guess0;
    double eta1 = input.guess1;

    double zEta0 = (input.leftValue - input.alpha * eta0) / input.beta;
    Trajectory tr0 = solveSystemRungeKutta(input.a, input.b, eta0, zEta0, h);
    double phi0 = input.delta * tr0.y.back() + input.gamma * tr0.z.back() - input.rightValue;

    double zEta1 = (input.leftValue - input.alpha * eta1) / input.beta;
    Trajectory tr1 = solveSystemRungeKutta(input.a, input.b, eta1, zEta1, h);
    double phi1 = input.delta * tr1.y.back() + input.gamma * tr1.z.back() - input.rightValue;

    log << "eta0 = " << eta0 << ", Phi(eta0) = " << phi0 << "\n";
    log << "eta1 = " << eta1 << ", Phi(eta1) = " << phi1 << "\n";

    int iter = 0;
    while (iter < MAX_ITER) {
        if (std::fabs(phi1 - phi0) < 1e-12) {
            throw std::runtime_error("В методе стрельбы возникло деление на ноль в формуле секущих");
        }

        const double eta2 = eta1 - (eta1 - eta0) * phi1 / (phi1 - phi0);
        const double zEta2 = (input.leftValue - input.alpha * eta2) / input.beta;
        Trajectory tr2 = solveSystemRungeKutta(input.a, input.b, eta2, zEta2, h);
        const double phi2 = input.delta * tr2.y.back() + input.gamma * tr2.z.back() - input.rightValue;
```

```

log << "k = " << iter
    << ", eta_k+1 = " << eta2
    << ", y(b) = " << tr2.y.back()
    << ", z(b) = " << tr2.z.back()
    << ", Phi(eta_k+1) = " << phi2
    << ", |eta_k+1 - eta_k| = " << std::fabs(eta2 - eta1)
    << "\n";

if (std::fabs(phi2) < input.eps || std::fabs(eta2 - eta1) < input.eps) {
    return {tr2.x, tr2.y, iter + 1, std::fabs(phi2)};
}

eta0 = eta1;
phi0 = phi1;
eta1 = eta2;
phi1 = phi2;
++iter;
}

throw std::runtime_error("Метод стрельбы не сошёлся");
}

```

Конечно-разностный метод:

```

BvpSolution solveFiniteDifference(const BvpInput& input, double h, std::ostream& log) {
    int n = computeStepCount(input.a, input.b, h);
    int size = n + 1;

    std::vector<double> x(size);
    for (int i = 0; i <= n; ++i) {
        x[i] = input.a + i * h;
    }

    std::vector<double> aDiag(size - 1, 0.0);
    std::vector<double> bDiag(size, 0.0);
    std::vector<double> cDiag(size - 1, 0.0);
    std::vector<double> dDiag(size, 0.0);

    log << "=== Конечно-разностный метод, h = " << h << " ===\n";

    bDiag[0] = input.alpha * h - input.beta;
    cDiag[0] = input.beta;
    dDiag[0] = input.leftValue * h;
    log << "Граничное условие слева: (" << bDiag[0] << ") * y0 + ("
        << cDiag[0] << ") * y1 = " << dDiag[0] << "\n";

    for (int i = 1; i < n; ++i) {
        double xi = x[i];
        aDiag[i - 1] = 1.0 - p(xi) * h / 2.0;
        bDiag[i] = q(xi) * h * h - 2.0;
        cDiag[i] = 1.0 + p(xi) * h / 2.0;
        dDiag[i] = f(xi) * h * h;

        log << "i = " << i
            << ": A = " << aDiag[i - 1]
            << ", B = " << bDiag[i]
            << ", C = " << cDiag[i]
            << ", D = " << dDiag[i]
            << "\n";
    }

    aDiag[n - 1] = -input.gamma;
    bDiag[n] = input.delta * h + input.gamma;
    dDiag[n] = input.rightValue * h;
    log << "Граничное условие справа: (" << aDiag[n - 1] << ") * y[n-1] + ("
        << bDiag[n] << ") * y[n] = " << dDiag[n] << "\n";

    std::vector<double> y = tridiagSolve(aDiag, bDiag, cDiag, dDiag, log);
}

```

```

    double residual = std::fabs(input.alpha * y.front() + input.beta * (y[1] - y[0]) / h
- input.leftValue)
    + std::fabs(input.delta * y.back() + input.gamma * (y.back() - y[n - 1]) / h -
input.rightValue);

    return {x, y, 1, residual};
}

```

Результат:

=== Задача 4.2. Решение краевой задачи ===

Точное решение: $y = e^x / x$
Интервал: [1.0000000000, 2.0000000000]
h1 = 0.1000000000, h2 = 0.0500000000

=== Метод стрельбы (h1) ===

x	y	y_exact	y - y_exact
1.0000000000	2.7182801670	2.7182818285	0.0000016614
1.1000000000	2.7310610406	2.7310600218	0.0000010189
1.2000000000	2.7667663613	2.7667641023	0.0000022591
1.3000000000	2.8225386579	2.8225358982	0.0000027598
1.4000000000	2.8965742783	2.8965714049	0.0000028735
1.5000000000	2.9877954985	2.9877927136	0.0000027849
1.6000000000	3.0956478591	3.0956452652	0.0000025939
1.7000000000	3.2199714089	3.2199690540	0.0000023550
1.8000000000	3.3609173561	3.3609152580	0.0000020981
1.9000000000	3.5188936512	3.5188918117	0.0000018394
2.0000000000	3.6945296365	3.6945280495	0.0000015871

iter = 1

Невязка граничного условия = 0.0000000000

=== Метод стрельбы (h2) ===

x	y	y_exact	y - y_exact
1.0000000000	2.7182817290	2.7182818285	0.0000000995
1.0500000000	2.7215724849	2.7215724934	0.0000000085
1.1000000000	2.7310600762	2.7310600218	0.0000000544
1.1500000000	2.7462548015	2.7462547041	0.0000000974
1.2000000000	2.7667642284	2.7667641023	0.0000001262
1.2500000000	2.7922745105	2.7922743660	0.0000001446
1.3000000000	2.8225360537	2.8225358982	0.0000001555
1.3500000000	2.8573524060	2.8573522450	0.0000001610
1.4000000000	2.8965715674	2.8965714049	0.0000001626
1.4500000000	2.9400791372	2.9400789760	0.0000001612
1.5000000000	2.9877928714	2.9877927136	0.0000001579
1.5500000000	3.0396583353	3.0396581823	0.0000001530
1.6000000000	3.0956454124	3.0956452652	0.0000001472
1.6500000000	3.1557454904	3.1557453498	0.0000001406
1.7000000000	3.2199691876	3.2199690540	0.0000001336
1.7500000000	3.2883445126	3.2883443863	0.0000001264
1.8000000000	3.3609153770	3.3609152580	0.0000001189
1.8500000000	3.4377403940	3.4377402825	0.0000001115
1.9000000000	3.5188919158	3.5188918117	0.0000001041
1.9500000000	3.6044552663	3.6044551695	0.0000000968
2.0000000000	3.6945281390	3.6945280495	0.0000000895

iter = 1

Невязка граничного условия = 0.0000000000

=== Конечно-разностный метод (h1) ===

x	y	y_exact	y - y_exact
1.0000000000	2.8268432553	2.7182818285	0.1085614269
1.1000000000	2.8268432553	2.7310600218	0.0957832336
1.2000000000	2.8527559852	2.7667641023	0.0859918829
1.3000000000	2.9010152734	2.8225358982	0.0784793752
1.4000000000	2.9693183765	2.8965714049	0.0727469716
1.5000000000	3.0562280374	2.9877927136	0.0684353238

1.6000000000	3.1609261285	3.0956452652	0.0652808633
1.7000000000	3.2830566901	3.2199690540	0.0630876362
1.8000000000	3.4226238359	3.3609152580	0.0617085779
1.9000000000	3.5799245605	3.5188918117	0.0610327488
2.0000000000	3.7555044960	3.6945280495	0.0609764465

Невязка граничных условий = 0.0000000000

=== Конечно-разностный метод (h2) ===

x	y	y_exact	y - y_exact
1.0000000000	2.7708745620	2.7182818285	0.0525927335
1.0500000000	2.7708745620	2.7215724934	0.0493020686
1.1000000000	2.7774868763	2.7310600218	0.0464268545
1.1500000000	2.7901660231	2.7462547041	0.0439113191
1.2000000000	2.8084733472	2.7667641023	0.0417092449
1.2500000000	2.8320564214	2.7922743660	0.0397820554
1.3000000000	2.8606332409	2.8225358982	0.0380973427
1.3500000000	2.8939799686	2.8573522450	0.0366277236
1.4000000000	2.9319213460	2.8965714049	0.0353499411
1.4500000000	2.9743231283	2.9400789760	0.0342441523
1.5000000000	3.0210860726	2.9877927136	0.0332933590
1.5500000000	3.0721411320	3.0396581823	0.0324829496
1.6000000000	3.1274455919	3.0956452652	0.0318003267
1.6500000000	3.1869799527	3.1557453498	0.0312346029
1.7000000000	3.2507454054	3.2199690540	0.0307763515
1.7500000000	3.3187617854	3.2883443863	0.0304173991
1.8000000000	3.3910659124	3.3609152580	0.0301506544
1.8500000000	3.4677102470	3.4377402825	0.0299699645
1.9000000000	3.5487618061	3.5188918117	0.0298699943
1.9500000000	3.6343012947	3.6044551695	0.0298461251
2.0000000000	3.7244224182	3.6945280495	0.0298943688

Невязка граничных условий = 0.0000000000

=== Оценка погрешности ===

Метод стрельбы:

max y_h - y_exact	= 0.0000028735
max y_h/2 - y_exact	= 0.0000001626
Runge-Romberg estimate	= 0.0000001807

Конечно-разностный метод:

max y_h - y_exact	= 0.1085614269
max y_h/2 - y_exact	= 0.0525927335
Runge-Romberg estimate	= 0.0559686934